

Projekt: Software Engineering

Projektdokumentation

MietManager

Portfolio von Michael Bangas

Matrikelnummer: IU14132885

Kursbezeichnung: Projekt: Software Engineering (DLMCSPSE01_D)

Studiengang: M.Sc. Informatik

IU Internationale Hochschule

25. September 2026

Inhaltsverzeichnis

1 Projektübersicht	1
1.1 Problemstellung und Motivation	1
1.2 Zielgruppe und Kundennutzen	1
1.3 Funktionsumfang	2
1.4 Angestrebtes Ergebnis	2
2 Theoretische Grundlagen	2
2.1 Risikomanagement in Softwareprojekten	2
2.2 Anforderungspriorisierung nach MoSCoW	3
2.3 Iteratives und inkrementelles Vorgehen	3
3 Risikomanagement	3
4 MoSCoW-Priorisierung	4
5 Zeitplanung	5
5.1 Woche 1 – Konzeptionsphase, Teil 1 (27.07.–02.08.2026)	6
5.2 Woche 2 – Konzeptionsphase, Teil 2 (03.08.–09.08.2026)	6
5.3 Woche 3 – Erarbeitungs-/Reflexionsphase (10.08.–16.08.2026)	6
5.4 Woche 4 – Finalisierungsphase (17.08.–23.08.2026)	7
6 GitHub-Repository	7
7 Benutzeranleitung	7
7.1 Installation und Start	7
7.2 Bedienung	8
8 Fazit	9
Literaturverzeichnis	11

Abbildungsverzeichnis

5.1 Projektzeitplan	6
-------------------------------	---

Tabellenverzeichnis

3.1	Risikomanagement	4
4.1	MoSCoW-Priorisierung des Funktionsumfangs	5

1 Projektübersicht

Der folgende Abschnitt beschreibt zunächst das zugrunde liegende Managementproblem privater Kleinvermieter, bevor darauf aufbauend die im Rahmen dieses Projekts entwickelte Anwendung MietManager mit Zielgruppe und Kundennutzen, geplantem Funktionsumfang sowie angestrebtem Ergebnis vorgestellt wird.

1.1 Problemstellung und Motivation

Private Kleinvermieter mit wenigen vermieteten Wohneinheiten stehen jährlich vor der Aufgabe, aus den anfallenden Betriebskosten eine korrekte Nebenkostenabrechnung je Mietvertrag zu erstellen. Bei manueller Bearbeitung mit Tabellenkalkulation oder auf Papier ist dieser Prozess fehleranfällig und zeitaufwendig. Kostenpositionen müssen nach dem vereinbarten Umlageschlüssel, etwa Fläche, Personenzahl oder Anzahl Einheiten, korrekt auf die einzelnen Mietverträge verteilt, den bereits geleisteten Vorauszahlungen gegenübergestellt und abschließend in einem versandfertigen Schreiben an die Mieter kommuniziert werden. Anders als größere Hausverwaltungen verfügen private Kleinvermieter dafür in der Regel über keine spezialisierte Software. Kommerzielle Hausverwaltungslösungen sind meist auf einen deutlich größeren Immobilienbestand ausgelegt. Diese Lücke zwischen dem tatsächlichen Bedarf privater Kleinvermieter und dem bestehenden Softwareangebot motiviert das vorliegende Projekt. Im Rahmen des Projekts entsteht die Desktop-Anwendung MietManager, die Immobilien, Mieteinheiten, Mieter und Mietverträge verwaltet und als Kernprozess automatisiert eine Nebenkostenabrechnung erstellt.

Daraus ergibt sich die für dieses Projekt leitende Projektfrage. „Inwiefern lässt sich die bislang manuelle, fehleranfällige Nebenkostenabrechnung privater Kleinvermieter durch eine automatisierte Desktop-Anwendung effizienter und fehlerfreier gestalten als mit Tabellenkalkulation oder Papier, und welche Anforderungen ergeben sich daraus für Datenmodell, Architektur und Benutzerschnittstelle?“

1.2 Zielgruppe und Kundennutzen

Zielgruppe sind private Vermieter mit dem in Abschnitt 1.1 beschriebenen Bedarf. Gegenüber einer Tabellenkalkulation liegt der zentrale Mehrwert darin, dass MietManager die Nebenkostenabrechnung nach frei wählbarem Umlageschlüssel, etwa Fläche, Personen oder Anzahl Einheiten, je Kostenart und Mietvertrag automatisch berechnet und dadurch die in der Problemstellung beschriebenen Rechenfehler sowie den Zeitaufwand manueller Berechnung vermeidet. Die Anwendung exportiert das Ergebnis direkt als versandfertigen, personalisierten Geschäftsbrief mit Briefkopf, persönlicher Anrede, Kostenaufschlüsselung, Saldo und Grußformel. Dazu hinterlegt der Vermieter seine eigenen

Stammdaten einmalig in einem Profil, das die Anwendung automatisch als Absender verwendet. Die erstellte Nebenkostenabrechnung kann er anschließend manuell per E-Mail versenden oder ausdrucken und postalisch verschicken.

1.3 Funktionsumfang

Im Kern verwaltet die Anwendung Immobilien sowie Mieter und Mietverträge und prüft dabei automatisch, ob sich Mietverträge derselben Mieteinheit zeitlich überschneiden. Darauf aufbauend erfasst sie die Kostenpositionen einer Immobilie je Abrechnungszeitraum und berechnet daraus die Nebenkostenabrechnung, die sich als versandfertiger PDF-Brief exportieren lässt. Das dafür benötigte Vermieter-Profil lässt sich als Briefkopf-Absender konfigurieren. Zusätzlich zeigt ein Dashboard Kennzahlen zu Immobilien, Einheiten, Leerstand, aktiven Verträgen und Mieteinnahmen. Bewusst nicht Teil dieses Funktionsumfangs sind angrenzende Problemfelder wie Zahlungs- und Mahnwesen, Mehrbenutzerbetrieb und eine mobile Anwendung. Diese Abgrenzung ergibt sich aus der in Abschnitt 1.1 beschriebenen Zielgruppe privater Kleinvermieter mit wenigen Wohneinheiten, für die eine Einzelplatz-Desktopanwendung ausreicht, und wird in der MoSCoW-Priorisierung in Abschnitt 4 im Detail begründet.

1.4 Angestrebtes Ergebnis

Das Projekt strebt eine funktionsfähige Desktop-Anwendung an, die alle in der Anforderungserhebung definierten funktionalen und nichtfunktionalen Anforderungen erfüllt, mit realistischen Testdaten vorbefüllt ist und deren Kernprozess, die automatisierte Nebenkostenabrechnung, sich direkt nach dem Start nachvollziehen lässt.

2 Theoretische Grundlagen

Dieser Abschnitt führt die dem Projektvorgehen zugrunde liegenden Konzepte theoretisch ein, bevor die Abschnitte Risikomanagement, MoSCoW-Priorisierung und Zeitplanung sie konkret auf MietManager anwenden.

2.1 Risikomanagement in Softwareprojekten

Risikomanagement in Softwareprojekten zielt darauf ab, potenzielle Probleme frühzeitig zu erkennen und aktiv zu steuern, statt erst auf ihr Eintreten zu reagieren. Boehm schlägt dazu vor, jedes Risiko anhand zweier Achsen zu bewerten. Diese sind die Eintrittswahrscheinlichkeit und die Schwere der

Auswirkung bei Eintritt (Boehm, 1991, S. 37). Diese Zwei-Achsen-Bewertung unterscheidet sich von einer bloßen, unstrukturierten Liste möglicher Probleme dadurch, dass sie eine begründete Priorisierung der Gegenmaßnahmen erlaubt. Risiken mit hoher Wahrscheinlichkeit und hoher Auswirkung erfordern vorrangig eine Gegenmaßnahme, während gering ausgeprägte Risiken auch unbehandelt tragbar sein können. Abschnitt 3 wendet dieses Bewertungsschema auf die Risiken des vorliegenden Projekts an.

2.2 Anforderungspriorisierung nach MoSCoW

Die MoSCoW-Methode ist ein Verfahren aus dem Anforderungsmanagement, das Anforderungen in die vier Kategorien Muss (Must have), Soll (Should have), Kann (Could have) und Verzicht (Won't have) einteilt (Ahmad et al., 2017, S. 433). Im Unterschied zu einer einfachen numerischen Priorisierung trifft jede Kategorie eine verbindliche Aussage über den Lieferumfang. Muss-Anforderungen sind für ein erfolgreiches Projektergebnis zwingend erforderlich, während Kann-Anforderungen bei Zeitdruck ohne Gefährdung des Projekterfolgs entfallen können. Gerade unter der in Abschnitt 3 identifizierten Zeitknappheit erlaubt diese Abstufung, den Funktionsumfang im Projektverlauf kontrolliert zu reduzieren, statt unter Zeitdruck an allen Anforderungen gleichermaßen Abstriche machen zu müssen. Abschnitt 4 wendet die Methode auf den Funktionsumfang von MietManager an.

2.3 Iteratives und inkrementelles Vorgehen

Iterative und inkrementelle Softwareentwicklung liefert den Projektumfang nicht in einem einzigen, vorab vollständig geplanten Durchlauf, sondern in aufeinanderfolgenden Zwischenergebnissen, die jeweils überprüft und bei Bedarf angepasst werden, bevor der nächste Abschnitt beginnt. Im Unterschied zu einem Wasserfallmodell, bei dem Fehler in frühen Phasen oft erst spät im Projekt sichtbar werden, lassen sich Rückmeldungen zu einem Zwischenergebnis so bereits vor Abschluss des Gesamtprojekts berücksichtigen. Abschnitt 5 beschreibt, wie dieses Vorgehen über drei Portfolio-Phasen mit jeweils eingeholtem Tutor-Feedback konkret umgesetzt wird.

3 Risikomanagement

Tabelle 3.1 wendet das in Abschnitt 2.1 eingeführte Bewertungsschema aus Eintrittswahrscheinlichkeit und Auswirkung auf die Risiken des vorliegenden Projekts an. Die beiden am höchsten eingestufteten Risiken, R2 (Wartezeit auf Tutor-Feedback) und R5 (Zeitknappheit durch Studium und Beruf), sind beide unmittelbare Folgen der knappen Gesamtdauer von vier Wochen, und werden daher durch dieselbe übergeordnete Gegenmaßnahme adressiert. Die Portfolioteile werden jeweils so früh

wie möglich eingereicht, was sowohl Pufferzeit für Feedback schafft als auch die in Abschnitt 2.2 eingeführte MoSCoW-Priorisierung nutzbar macht, um den Funktionsumfang bei Bedarf kontrolliert zu reduzieren. Für keines der sechs identifizierten Risiken wird auf eine Gegenmaßnahme verzichtet, da laut dem in Abschnitt 2.1 eingeführten Bewertungsschema erst die Kombination aus niedriger Wahrscheinlichkeit und niedriger Auswirkung ein Tolerieren ohne Gegenmaßnahme rechtfertigen würde. Selbst R6 (Datenverlust) erfordert trotz niedriger Eintrittswahrscheinlichkeit wegen seiner hohen Auswirkung eine Gegenmaßnahme.

#	Risiko	Wahrsch.	Auswirkung	Gegenmaßnahme
R1	Zu viele Features geplant	mittel	hoch	MoSCoW-Priorisierung
R2	Wartezeit auf Tutor-Feedback verzögert Zeitplan	hoch	hoch	Portfolioteile 1 und 2 möglichst früh statt am letzten Tag einreichen, um innerhalb der Woche noch auf Feedback reagieren zu können
R3	Berechnungslogik der Nebenkostenabrechnung unterschätzt	mittel	hoch	Rechenprototyp früh in Woche 3 bauen, bevor die GUI angebunden wird
R4	Packaging-Probleme	mittel	mittel	PyInstaller bereits in Woche 3 testen, nicht erst am Ende
R5	Zeitknappheit durch Studium/Beruf bei nur 4 Wochen Gesamtdauer	hoch	hoch	Klare Priorisierung nach MoSCoW, keine Pufferzeit für Nice-to-have-Features einplanen
R6	Datenverlust	niedrig	hoch	Bereits zu Beginn der Implementierung GitHub mit regelmäßigen, kleinteiligen Commits nutzen

Tabelle 3.1: Risikomanagement

4 MoSCoW-Priorisierung

Abschnitt 2.2 führt die MoSCoW-Methode als Reaktion auf das in Abschnitt 3 identifizierte Risiko von zu vielen geplanten Features ein. Tabelle 4.1 wendet sie auf den Funktionsumfang von MietManager an. Die Muss-Anforderungen entsprechen dabei genau den User Stories FA-01 bis FA-07 aus dem Anforderungsdokument, da ohne sie der in der Problemstellung beschriebene Kernnutzen, die automatisierte Nebenkostenabrechnung, nicht realisiert wäre. Das Dashboard (FA-08) wurde dagegen als Soll statt als Muss eingestuft, weil es den bereits vollständigen Kernprozess lediglich um eine zusätzliche Übersicht ergänzt, ohne selbst Voraussetzung für die Abrechnungserstellung zu sein. E-Mail-Versand und Kennzahlen-Export wurden als Kann eingestuft und bewusst aus dem Muss-Umfang ausgeschlossen, da beide den bestehenden Kernprozess nur bequemer, aber nicht grundsätzlich anders nutzbar machen.

Kategorie	Umfang
Muss (Must have)	Immobilien- und Mieteinheitenverwaltung, Mieterverwaltung, Mietvertragsverwaltung mit Überlappungsprüfung, Kostenpositionserfassung, automatische Berechnung der Nebenkostenabrechnung, PDF-Export als versandfertiger Geschäftsbrief, Vermieterprofil als Briefkopf-Absender (FA-01 bis FA-07).
Soll (Should have)	Dashboard mit Übersicht zu Immobilien, Einheiten, Leerstand, aktiven Verträgen sowie Mieteinnahmen (FA-08).
Kann (Could have)	Automatischer E-Mail-Versand der Abrechnung direkt aus der Anwendung heraus, CSV-/Excel-Export der Kennzahlen.
Verzicht (Won't have)	Zahlungs- und Mahnwesen, Mehrbenutzerbetrieb, mobile Anwendung.

Tabelle 4.1: MoSCoW-Priorisierung des Funktionsumfangs

5 Zeitplanung

Abschnitt 2.3 führt das iterative und inkrementelle Vorgehen ein, dem das Projekt folgt. Die drei Portfolio-Phasen liefern jeweils ein abgeschlossenes Zwischenergebnis, zu dem vor der nächsten Phase Tutor-Feedback eingeholt und eingearbeitet wird. Das Projekt ist auf eine Gesamtdauer von vier Wochen angesetzt (Mo. 27.07.2026 bis So. 23.08.2026). Die Konzeptionsphase erhält davon zwei Kalenderwochen, da hier mit Projektdokumentation, Anforderungsdokument und Spezifikationsdokument die meisten Dokumente entstehen und eine sorgfältige Planung für den weiteren Projektverlauf entscheidend ist. Die Erarbeitungs- und Finalisierungsphase erhalten je eine Kalenderwoche. Trotz der zusätzlichen Woche bleibt der Puffer insgesamt knapp (siehe Risiken R2 und R5), weshalb die einzelnen Portfolioteile jeweils so früh wie möglich fertiggestellt werden, um noch Zeit für Tutor-Feedback und Korrekturen zu haben. Abbildung 5.1 zeigt den Projektzeitplan als Gantt-Diagramm. Anschließend werden die vier Wochen inhaltlich beschrieben. Der Projektfortschritt lässt sich an den drei Meilensteinen der Portfolioabgabe ablesen. Der Einreichung von Portfolioteil 1 (Konzeptionsphase) am 09.08.2026, von Portfolioteil 2 (Erarbeitungs-/Reflexionsphase) am 16.08.2026 und von Portfolioteil 3 (Finalisierungsphase) am 23.08.2026.

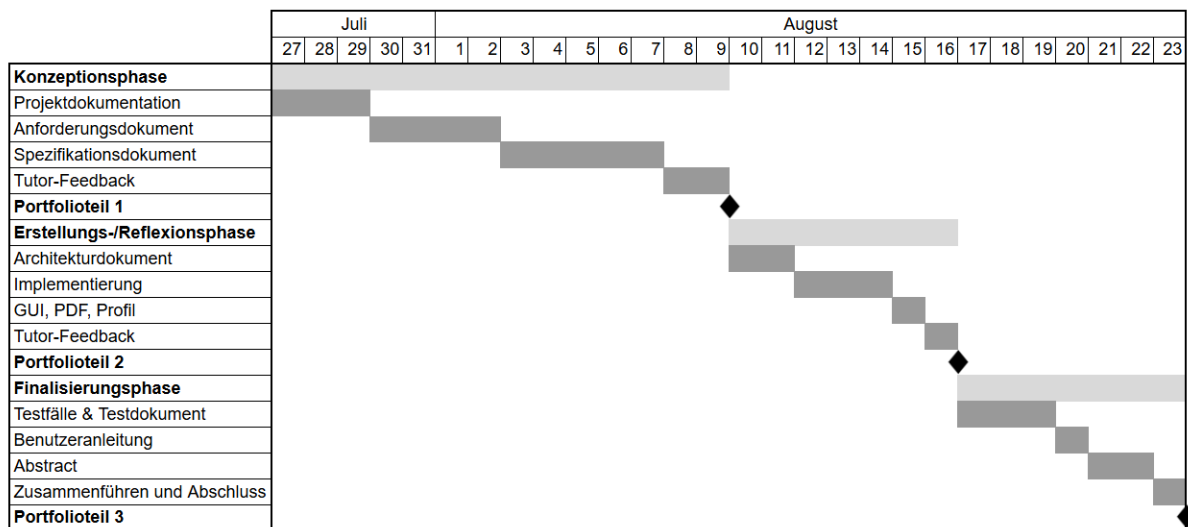


Abbildung 5.1: Projektzeitplan

5.1 Woche 1 – Konzeptionsphase, Teil 1 (27.07.–02.08.2026)

In der ersten Woche werden zunächst Projektübersicht, Risikomanagement, MoSCoW-Priorisierung und Zeitplanung als Grundlage der Projektdokumentation erstellt. Anschließend werden für das Anforderungsdokument die Stakeholder identifiziert, User Stories gesammelt, die nichtfunktionalen Anforderungen definiert und die funktionalen Anforderungen als UML-Use-Case-Diagramm visualisiert.

5.2 Woche 2 – Konzeptionsphase, Teil 2 (03.08.–09.08.2026)

In der zweiten Woche entsteht das Spezifikationsdokument. Das Datenmodell wird erarbeitet und als UML-Klassendiagramm visualisiert, der Kernprozess „Nebenkostenabrechnung erstellen“ als UML-Aktivitätsdiagramm dargestellt sowie Geschäftsregeln, Systemschnittstellen und Benutzerschnittstellen inklusive GUI-Mockups beschrieben. Vor Abgabe wird Tutor-Feedback zu allen drei Dokumenten der Konzeptionsphase eingeholt. Das Ergebnis der Konzeptionsphase ist die Einreichung von Portfolioteil 1.

5.3 Woche 3 – Erarbeitungs-/Reflexionsphase (10.08.–16.08.2026)

In Woche 3 wird das Architekturdokument erstellt und die Struktur des GitHub-Repositorys aufgesetzt. Im Zentrum der Woche steht anschließend die Implementierung. Datenmodell und Datenbank, CRUD-Funktionen, die Berechnungslogik der Nebenkostenabrechnung, die grafische Oberfläche, der PDF-Export sowie das Vermieter-Profil werden umgesetzt. Vor der Abgabe wird Tutor-Feedback eingeholt. Die Erarbeitungs- und Reflexionsphase endet mit der Einreichung von Portfolioteil 2.

5.4 Woche 4 – Finalisierungsphase (17.08.–23.08.2026)

Die vierte Woche beginnt mit der Einarbeitung des Tutor-Feedbacks aus Phase 2. Anschließend werden Testfälle geschrieben und im Testdokument protokolliert sowie die Benutzeranleitung verfasst. Zum Abschluss entsteht das zweiseitige Abstract, bevor alle Dokumente zusammengeführt und eine ZIP-Datei aus dem Inhalt des GitHub-Repositorys erzeugt werden. Die Finalisierungsphase signalisiert den Abschluss des Projekts und endet mit der Einreichung von Portfolioteil 3.

6 GitHub-Repository

Der vollständige Quellcode, die Tests sowie alle zur Ausführung der Anwendung notwendigen Dateien sind im folgenden GitHub-Repository abgelegt: https://github.com/Michael5601/PSE_Abgabe_MietManager

7 Benutzeranleitung

Der folgende Abschnitt beschreibt zunächst Installation und Start von MietManager, bevor die Bedienung der wichtigsten Tabs anhand des typischen Arbeitsablaufs erläutert wird.

7.1 Installation und Start

Für die Nutzung von MietManager auf einem Windows-10/11-Rechner gibt es zwei Wege.

Am einfachsten ist der Start über die im GitHub-Repository unter `dist/MietManager.exe` bereitgestellte eigenständige Anwendung. Sie wurde mit PyInstaller aus dem Quellcode gebaut und bündelt bereits sämtliche Laufzeitabhängigkeiten. Auf dem Zielrechner müssen daher weder Python noch zusätzliche Bibliotheken installiert werden, ein Doppelklick auf die Datei startet die Anwendung direkt.

Alternativ lässt sich MietManager aus dem Quellcode ausführen. Dafür wird Python 3.14 oder neuer benötigt. Sind auf dem Rechner mehrere Python-Versionen installiert, sollte die virtuelle Umgebung über den Windows-Python-Launcher mit expliziter Versionsangabe angelegt werden, da der einfache Befehl `python` je nach PATH-Reihenfolge auch eine ältere Version auflösen kann. Folgende Befehle klonen das Repository, wechseln in den dabei neu angelegten Projektordner und richten dort eine virtuelle Umgebung mit allen Abhängigkeiten ein.

- `git clone https://github.com/Michael5601/PSE_Abgabe_MietManager.git`
- `cd PSE_Abgabe_MietManager`

- `py -3.14 -m venv .venv`
- `.venv\Scripts\activate`
- `pip install -e ".[dev]"`

Der Befehl `mietmanager` startet anschließend die Anwendung. Dieser Weg eignet sich vor allem, um den Quellcode nachzuvollziehen, die automatisierten Tests mit `pytest` auszuführen oder mit `python scripts/performance_benchmark.py` den im Testdokument beschriebenen Performance-Benchmark gegen eine an der dokumentierten Skalierungsgrenze geseedete Datenbank zu wiederholen.

Beim ersten Start legt MietManager unabhängig vom gewählten Weg automatisch eine lokale SQLite-Datenbank unter `%USERPROFILE%\mietmanager\mietmanager.db` an und befüllt sie mit realistischen Beispieldaten zu Immobilien, Mietern, Mietverträgen und Kostenpositionen. Dadurch lässt sich der in Abschnitt 1.1 beschriebene Kernprozess, das Erstellen einer Nebenkostenabrechnung, direkt nach dem Start nachvollziehen, ohne zunächst manuell Daten anlegen zu müssen. Bei jedem weiteren Start greift die Anwendung auf dieselbe Datenbank zu, sodass alle Eingaben dauerhaft erhalten bleiben.

7.2 Bedienung

Die Anwendung gliedert sich in sechs Tabs, die in der Reihenfolge des typischen Arbeitsablaufs angeordnet sind: Dashboard, Immobilien, Mieter, Mietverträge, Kostenpositionen und Nebenkostenabrechnung. Vor der ersten Nutzung sollte über den Profil-Button oben rechts das Vermieterprofil mit Name, Straße, PLZ und Ort ausgefüllt werden, da diese Angaben als Absender jeder exportierten Nebenkostenabrechnung dienen. Im Tab Immobilien legt „Neue Immobilie“ zunächst das Gebäude mit Bezeichnung und Adresse an. Durch Auswahl der Immobilie und „Neue Mieteinheit“ lassen sich anschließend die einzelnen Wohneinheiten mit Fläche und Zimmerzahl ergänzen. Im Tab Mieter werden die Kontaktdaten der Mietparteien erfasst. Im Tab Mietverträge verknüpft ein neuer Mietvertrag eine Mieteinheit mit einem Mieter und legt Beginn, Kaltmiete, Nebenkosten-Vorauszahlung, Kautions- und Personenzahl fest. Überschneidet sich der gewählte Zeitraum mit einem bestehenden Mietvertrag derselben Mieteinheit, lehnt die Anwendung das Speichern automatisch ab. Im Tab Kostenpositionen werden die anfallenden Betriebskosten je Immobilie, Zeitraum und Umlageschlüssel erfasst, etwa Grundsteuer nach Fläche oder Müllabfuhr nach Personenzahl. Liegen für eine Immobilie sowohl Kostenpositionen als auch aktive Mietverträge im gewünschten Abrechnungszeitraum vor, berechnet der Tab Nebenkostenabrechnung nach Auswahl von Immobilie und Zeitraum sowie einem Klick auf „Abrechnung erstellen“ die Kostenverteilung. Die Ergebnistabelle zeigt je Mietvertrag den Kostenanteil, die geleistete Vorauszahlung und den daraus resultierenden Saldo. Über „Als PDF exportieren...“

lässt sich die Abrechnung anschließend als versandfertiger, personalisierter Brief je Mietvertrag in einem frei wählbaren Zielordner speichern. Bereits erstellte Abrechnungen bleiben im Verlauf desselben Tabs einsehbar und lassen sich dort bei Bedarf auch wieder löschen. Das Dashboard fasst währenddessen jederzeit die wichtigsten Kennzahlen zu Immobilien, Einheiten, Leerstand, aktiven Verträgen und Mieteinnahmen zusammen.

8 Fazit

Dieses Dokument leitet ausgehend vom Managementproblem privater Kleinvermieter die Projektfrage, Zielgruppe und den Kundennutzen von MietManager her und grenzt den Funktionsumfang mittels MoSCoW-Priorisierung ab. Darauf aufbauend beschreibt das Anforderungsdokument Stakeholder sowie funktionale und nichtfunktionale Anforderungen, das Spezifikationsdokument konkretisiert Datenmodell und Geschäftsprozesse, das Architekturdokument begründet die gewählte Drei-Schichten-Architektur gegenüber den betrachteten Alternativen und das Testdokument weist die Erfüllung der Anforderungen anhand von 42 automatisierten und 28 manuellen Testfällen nach.

Die eingangs in Abschnitt 1.1 gestellte Projektfrage lässt sich auf dieser Grundlage positiv beantworten. Anstelle einer manuellen Verteilung der Kostenpositionen nach Umlageschlüssel und eines manuellen Abgleichs mit den geleisteten Vorauszahlungen, beides in der Problemstellung als fehleranfällige und zeitaufwendige Schritte beschrieben, übernimmt die implementierte Anwendung Berechnung, Abgleich und den Export als versandfertigen, personalisierten PDF-Geschäftsbrief vollständig automatisiert. Alle sieben als Muss eingestuft funktionalen Anforderungen sind ebenso umgesetzt wie die als Soll eingestufte Dashboard-Anforderung, beides ist durch das Testdokument mit 42 automatisierten und 28 manuellen Testfällen verifiziert, wodurch die Fehlerfreiheit der Berechnung reproduzierbar nachgewiesen ist. Welche Anforderungen sich daraus für Datenmodell, Architektur und Benutzerschnittstelle ergeben, beschreiben Anforderungs-, Spezifikations- und Architekturdokument im Detail, wie eingangs zusammengefasst. Eine Desktop-Anwendung kann private Kleinvermieter damit im beschriebenen Umfang tatsächlich dabei unterstützen, die bislang manuelle Nebenkostenabrechnung effizienter und fehlerfreier zu gestalten als mit Tabellenkalkulation oder Papier.

Aus den einzelnen Dokumenten lassen sich zwei Arten von Limitationen unterscheiden. Zum einen bestehen bewusst getroffene Scope-Entscheidungen, die keine technische Schuld darstellen, sondern die in der MoSCoW-Priorisierung begründete Abgrenzung auf die Zielgruppe privater Kleinvermieter widerspiegeln. Dazu zählen der Verzicht auf Zahlungs- und Mahnwesen, auf Mehrbenutzerbetrieb und auf eine mobile Anwendung. Zum anderen bestehen technische Grenzen, die aus den getroffenen Architekturentscheidungen resultieren. Die dateibasierte SQLite-Datenbank legt die in den nichtfunktionalen Anforderungen quantifizierte Skalierungsgrenze von etwa 20 Mieteinheiten und 200

Mietverträgen fest und schließt gleichzeitigen Mehrbenutzerzugriff aus. Eine Datensicherung ist bislang nur manuell durch Kopieren der Datenbankdatei möglich, eine automatisierte Backup-Funktion fehlt. Auch die im Testdokument begründete Entscheidung, die UI- und Datenzugriffsschicht nicht automatisiert, sondern nur durch manuelle Systemtests abzusichern, stellt eine bewusst eingegangene Grenze der Qualitätssicherung dar. Zwei der als Kann eingestuft Funktionen, der automatische E-Mail-Versand und der Kennzahlen-Export, wurden im Rahmen dieses Projekts nicht umgesetzt.

Aus dem Projektverlauf lassen sich mehrere Erkenntnisse für künftige Softwareprojekte ableiten. Die im Architekturdokument begründete Entscheidung für eine strikte Drei-Schichten-Architektur bestätigte sich als tragfähig, weil sie die Geschäftslogik unabhängig von der Benutzeroberfläche testbar machte, noch bevor die grafische Oberfläche überhaupt existierte. Umgekehrt zeigte sich, dass eine Zeitplanung, die vor allem die eigene Bearbeitungszeit kalkuliert, den tatsächlichen Kalenderaufwand systematisch unterschätzt, sobald die Bearbeitung wie in diesem Projekt nur in begrenzten Zeitfenstern neben Studium und Beruf möglich ist. Schließlich bestätigte sich, dass Usability-Schwächen wie die ursprünglich manuellen statt automatischen Aktualisierungen in der Oberfläche oft erst durch die tatsächliche Nutzung der fertigen Anwendung sichtbar werden und sich nicht vollständig durch Mock-ups in der Konzeptionsphase vorwegnehmen lassen.

Für eine Weiterentwicklung über den Rahmen dieses Portfolios hinaus ergeben sich daraus konkrete Handlungsempfehlungen. Kurzfristig ließen sich die bereits als Kann eingestuft Funktionen, der automatische E-Mail-Versand und der CSV-Export der Dashboard-Kennzahlen, ohne Änderung der bestehenden Architektur ergänzen. Mittelfristig sollte eine automatisierte Backup-Funktion die bislang manuelle Datensicherung ablösen, um dem in den nichtfunktionalen Anforderungen beschriebenen Risiko eines Datenverlusts weiter vorzubeugen. Sollte der adressierte Kundenstamm über die dokumentierte Skalierungsgrenze hinauswachsen oder Mehrbenutzerbetrieb erforderlich werden, müsste die dateibasierte SQLite-Anbindung durch eine Client-Server-Datenbank ersetzt werden, wofür die bereits bestehende Trennung von Service- und Datenzugriffsschicht eine geeignete Grundlage bietet. Auch die Testabdeckung ließe sich erweitern, etwa durch den Einsatz von Werkzeugen wie pytest-qt, um künftig auch die bislang nur manuell geprüfte Benutzeroberfläche automatisiert abzusichern.

Literaturverzeichnis

- Ahmad, K. S., Ahmad, N., Tahir, H., & Khan, S. (2017). Fuzzy_MoSCoW: A fuzzy based MoSCoW method for the prioritization of software requirements. *International Conference on Intelligent Computing, Instrumentation and Control Technologies (ICICICT)*, 433–437. <https://doi.org/10.1109/ICICICT1.2017.8342602>
- Boehm, B. W. (1991). Software risk management: principles and practices. *IEEE Software*, 8(1), 32–41. <https://doi.org/10.1109/52.62930>

Projekt: Software Engineering

Anforderungsdokument

MietManager

Portfolio von Michael Bangas

Matrikelnummer: IU14132885

Kursbezeichnung: Projekt: Software Engineering (DLMCSPSE01_D)

Studiengang: M.Sc. Informatik

IU Internationale Hochschule

25. September 2026

Inhaltsverzeichnis

1 Stakeholder	1
1.1 Vermieter (primärer Nutzer)	1
1.2 Mieter (indirekter Stakeholder)	1
1.3 Tutoren (im Rahmen des Kurses)	1
2 Funktionale Anforderungen	1
2.1 User Stories	1
2.2 UML-Use-Case-Diagramm	3
3 Nichtfunktionale Anforderungen	4
4 Glossar	5
5 Fazit	5
Literaturverzeichnis	6

Abbildungsverzeichnis

2.1 UML-Use-Case-Diagramm	4
-------------------------------------	---

Tabellenverzeichnis

2.1	User Stories mit Akzeptanzkriterien	2
4.1	Glossar	5

1 Stakeholder

Dieser Abschnitt identifiziert die am Projekt beteiligten Ziel- und Benutzergruppen und beschreibt ihre jeweilige Rolle sowie ihr Interesse an der Anwendung.

1.1 Vermieter (primärer Nutzer)

Private Kleinvermieter mit wenigen Wohneinheiten sind die primäre Zielgruppe von MietManager. Sie bedienen die Anwendung direkt, verwalten ihre Immobilien, Mieter und Mietverträge und erstellen die Nebenkostenabrechnung. Diese Gruppe verwaltet ihre Objekte bislang häufig mit Excel oder auf Papier und hat in der Regel keine spezialisierte Software im Einsatz.

1.2 Mieter (indirekter Stakeholder)

Mieter interagieren nicht direkt mit der Anwendung, sind aber indirekt betroffen. Sie erhalten die von MietManager erzeugte Nebenkostenabrechnung als personalisierten Geschäftsbrief. Die Verständlichkeit und Korrektheit der Abrechnung wirkt sich damit unmittelbar auf diese Gruppe aus, auch wenn sie kein Benutzerkonto oder keinen Zugriff auf die Software besitzt.

1.3 Tutoren (im Rahmen des Kurses)

Im Rahmen dieses Portfoliokurses sind die betreuenden Tutoren gemäß Aufgabenstellung als Zielgruppe bzw. Benutzer der zu erstellenden Anwendung zu betrachten. Sie bewerten die Anwendung aus Sicht eines potenziellen Vermieters und geben Feedback zu Bedienbarkeit und Funktionsumfang.

2 Funktionale Anforderungen

Dieser Abschnitt beschreibt die funktionalen Anforderungen an MietManager zunächst als priorisierte User Stories und visualisiert sie anschließend in einem UML-Use-Case-Diagramm.

2.1 User Stories

Tabelle 2.1 listet die funktionalen Anforderungen als User Stories im Format „Als <Rolle> möchte ich <Ziel>, um <Nutzen> zu erreichen.“ auf, ordnet jeder ein prüfbares Akzeptanzkriterium zu und priorisiert sie nach der MoSCoW-Methode.

Tabelle 2.1: User Stories mit Akzeptanzkriterien

ID	User Story	Akzeptanzkriterium	Priorität
FA-01	Als Vermieter möchte ich Immobilien und deren Mieteinheiten anlegen und pflegen, um meinen Immobilienbestand zentral zu verwalten.	Nach dem Anlegen einer Immobilie mit Bezeichnung und Adresse sowie mindestens einer Mieteinheit erscheinen beide in der Immobilienübersicht.	Muss
FA-02	Als Vermieter möchte ich Mieter mit ihren Kontaktdaten erfassen, um alle mieterbezogenen Informationen an einem Ort zu haben.	Nach der Eingabe von Name und Kontaktdaten erscheint der Mieter in der Mieterliste und kann bei der Anlage eines Mietvertrags ausgewählt werden.	Muss
FA-03	Als Vermieter möchte ich Mietverträge anlegen, die automatisch auf zeitliche Überschneidungen je Mieteinheit geprüft werden, um Doppelvermietungen zu vermeiden.	Beim Speichern eines Mietvertrags, dessen Zeitraum sich mit einem bestehenden Mietvertrag derselben Mieteinheit überschneidet, wird eine Fehlermeldung angezeigt und der Vertrag nicht gespeichert.	Muss
FA-04	Als Vermieter möchte ich Kostenpositionen je Immobilie und Zeitraum erfassen, um die Grundlage für die Nebenkostenabrechnung zu schaffen.	Nach der Eingabe von Kostenart, Betrag, Zeitraum und Umlageschlüssel erscheint die Kostenposition in der Liste der Immobilie.	Muss
FA-05	Als Vermieter möchte ich die Nebenkostenabrechnung automatisch nach einem frei wählbaren Umlageschlüssel berechnen lassen, um Zeit zu sparen und Rechenfehler zu vermeiden.	Beim Erstellen einer Abrechnung werden alle im gewählten Zeitraum aktiven Kostenpositionen gemäß ihrem Umlageschlüssel auf die aktiven Mietverträge verteilt. Die Summe der berechneten Kostenanteile entspricht dabei der Summe der zugrunde liegenden Kostenpositionen.	Muss
FA-06	Als Vermieter möchte ich die berechnete Nebenkostenabrechnung als versandfertigen, personalisierten PDF-Brief exportieren, um sie ohne Zwischenschritt an meine Mieter zu verschicken.	Nach Klick auf einen „Exportieren-Button“ wird je Mietvertrag eine PDF-Datei mit Briefkopf, persönlicher Anrede, Kostenaufschlüsselung, Saldo und Grußformel im gewählten Zielordner erzeugt.	Muss

Tabelle 2.1 – Fortsetzung

ID	User Story	Akzeptanzkriterium	Priorität
FA-07	Als Vermieter möchte ich meine eigenen Stammdaten einmalig in einem Profil hinterlegen, damit sie automatisch als Briefkopf verwendet werden.	Nach dem Ausfüllen und Speichern von Name, Straße, PLZ und Ort im Vermieterprofil erscheinen diese Daten als Absender im Briefkopf jeder neu exportierten Nebenkostenabrechnung. Bei unvollständigem Profil wird der Export mit einer Fehlermeldung verhindert.	Muss
FA-08	Als Vermieter möchte ich ein Dashboard mit den wichtigsten Kennzahlen sehen, um schnell einen Überblick über Leerstand und Mieteinnahmen zu erhalten.	Nach dem Anlegen oder Ändern einer Immobilie, Mieteinheit oder eines Mietvertrags zeigt das Dashboard beim nächsten Öffnen aktualisierte Kennzahlen zu Immobilien, Einheiten, Leerstand, aktiven Verträgen und Mieteinnahmen.	Soll

2.2 UML-Use-Case-Diagramm

Abbildung 2.1 visualisiert die funktionalen Anforderungen aus Sicht des Akteurs „Vermieter“. Da Mieter nicht direkt mit der Anwendung interagieren, treten sie im Use-Case-Diagramm nicht als Akteur auf (siehe Abschnitt 1.2). Der Anwendungsfall „Nebenkostenabrechnung erstellen“ entspricht User Story FA-05 und wird im Spezifikationsdokument auf Ebene der einzelnen Verarbeitungsschritte als UML-Aktivitätsdiagramm detailliert dargestellt. Die dabei verarbeiteten Geschäftsobjekte sind im zugehörigen UML-Klassendiagramm modelliert.

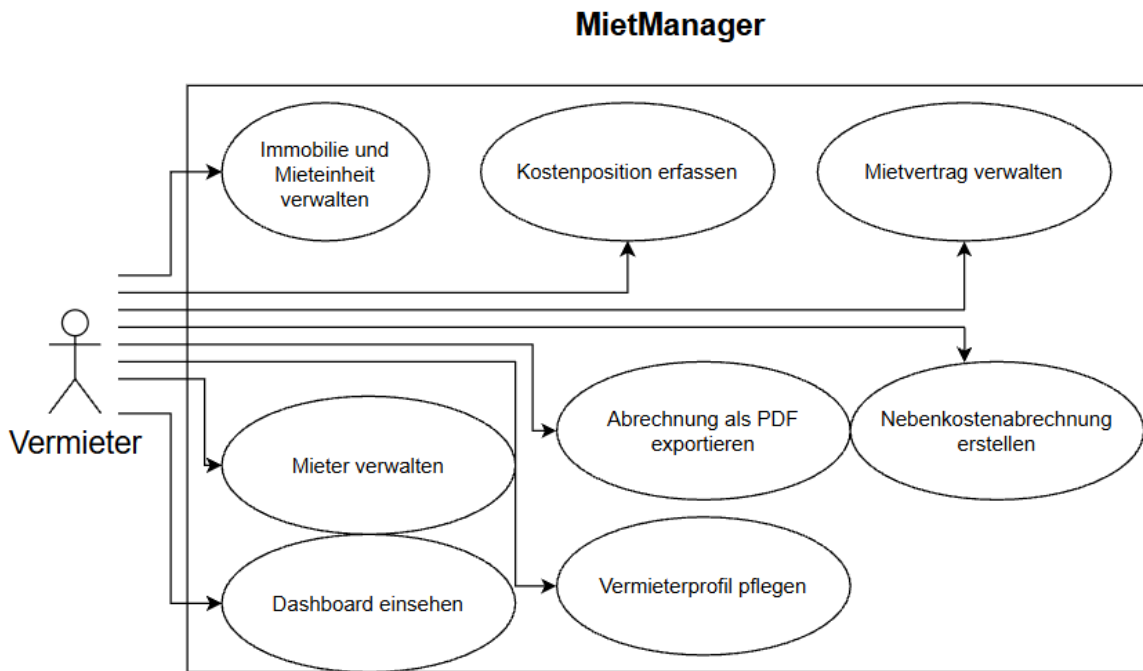


Abbildung 2.1: UML-Use-Case-Diagramm

3 Nichtfunktionale Anforderungen

Im Hinblick auf die Benutzerfreundlichkeit lässt sich der Kernprozess, das Erstellen einer Nebenkostenabrechnung, ohne Handbuch oder Einweisung in maximal drei Bedienschritten abschließen: Immobilie und Zeitraum auswählen, die Abrechnung erstellen und sie als PDF exportieren. Jeder Erfassungsdialog benennt zudem beim Speichern fehlende Pflichtfelder namentlich in einer Fehlermeldung, sodass Nutzer:innen ohne Rückfrage erkennen, was zu korrigieren ist. Beide Kriterien sind über die Testfälle TF-02, TF-10, TF-15, TF-21, TF-22, TF-23 und TF-25 des Testdokuments nachprüfbar. Bezüglich der Performance reagiert die Benutzeroberfläche bei einem Bestand von bis zu 20 Mieteinheiten innerhalb von 100 ms auf Benutzerinteraktionen. Dieser Schwellwert orientiert sich am in der Usability-Forschung etablierten Richtwert, ab dem Reaktionen von Nutzern als unmittelbar und nicht als Wartezeit wahrgenommen werden (Nielsen, 1993). Der Bestand von 20 Mieteinheiten orientiert sich an der in Abschnitt 1.1 beschriebenen Zielgruppe privater Kleinvermieter mit wenigen Wohneinheiten. Ab einem zweistelligen Wohnungsbestand wird die manuelle Verwaltung mit Excel oder auf Papier gemäß der Problemstellung zunehmend unübersichtlich, sodass genau in dieser Größenordnung der Kundennutzen von MietManager ansetzt. Hinsichtlich der Skalierbarkeit ist die Anwendung auf diesen Bestand von bis zu 20 Mieteinheiten ausgelegt, was bei realistischer Nutzung über mehrere Jahre einem Gesamtbestand von bis zu 200 Mietverträgen, inklusive beendeter Verträge, und bis zu 400 Kostenpositionen in der lokalen SQLite-Datenbank entspricht. Diese Werte sind be-

wusst als großzügige Obergrenze für einen privaten Kleinvermieter gewählt, die auch bei häufigerem Mieterwechsel und mehrjähriger Nutzung nicht überschritten werden dürfte. Darüber hinausgehende Bestände sowie gleichzeitiger Mehrbenutzerzugriff werden von der dateibasierten SQLite-Architektur bewusst nicht unterstützt, da laut MoSCoW-Priorisierung in der Projektdokumentation auf Mehrbenutzerfähigkeit verzichtet wird. Bezüglich der Zuverlässigkeit läuft jede Datenbankoperation als einzelne SQLite-Transaktion. Ein Commit schreibt dabei entweder vollständig oder gar nicht in die Datenbankdatei, wodurch bereits die Transaktionsgarantie von SQLite verhindert, dass ein bei einem Fehler abgebrochener Vorgang einen inkonsistenten Zwischenstand in der Datenbank hinterlässt. Da sämtliche Daten in einer einzelnen SQLite-Datei abgelegt werden, kann der Vermieter durch einfaches Kopieren dieser Datei jederzeit eine manuelle Sicherung anlegen und so einem Datenverlust vorbeugen. Für die Datensicherheit werden die Daten ausschließlich lokal in einer SQLite-Datenbank im Benutzerprofil gespeichert. Eine Übertragung an Dritte findet nicht statt. Hinsichtlich der Portabilität ist die Anwendung unter Windows 10/11 lauffähig, da diese Versionen zum Zeitpunkt der Projekterstellung die aktuell von Microsoft unterstützten Desktop-Betriebssystemversionen sind, ohne dass über die vom Kurs zugelassenen Standard-Laufzeitumgebungen hinaus zusätzliche Software installiert werden muss.

4 Glossar

Das folgende Glossar erläutert die im Projekt verwendeten Fachbegriffe und Abkürzungen.

Begriff	Erläuterung
Immobilie	Ein vermietetes Gebäude oder Grundstück, bestehend aus einer oder mehreren Mieteinheiten.
Mieteinheit	Einzelne vermietbare Einheit innerhalb einer Immobilie.
Umlageschlüssel	Regel, nach der die Kosten einer Kostenposition auf die Mieteinheiten einer Immobilie verteilt werden z.B. nach Fläche, Personenzahl oder Anzahl Einheiten.
Kostenposition	Eine einzelne erfasste Betriebskostenart einer Immobilie für einen bestimmten Zeitraum.
Nebenkostenabrechnung	Das aus den Kostenpositionen eines Zeitraums berechnete Ergebnis je Mietvertrag.
Vermieterprofil	Die Stammdaten des Vermieters, die als Absender auf der Nebenkostenabrechnung erscheinen.
MoSCoW-Priorisierung	Methode aus dem Anforderungsmanagement, die Anforderungen in Muss (Must have), Soll (Should have), Kann (Could have) und Verzicht (Won't have) einteilt.

Tabelle 4.1: Glossar

5 Fazit

Dieses Dokument legt mit den drei Stakeholdergruppen Vermieter, Mieter und Tutoren sowie acht funktionalen Anforderungen inklusive Akzeptanzkriterien und MoSCoW-Priorisierung, visualisiert im

UML-Use-Case-Diagramm, den funktionalen Rahmen von MietManager fest. Zusammen mit den nichtfunktionalen Anforderungen zu Benutzerfreundlichkeit, Performance, Skalierbarkeit, Zuverlässigkeit, Datensicherheit und Portabilität sowie dem Glossar legt es damit die Grundlage, auf der das Spezifikationsdokument das Datenmodell und die Geschäftsprozesse im Detail konkretisiert.

Literaturverzeichnis

Nielsen, J. (1993). *Response Times: The 3 Important Limits*. <https://www.nngroup.com/articles/response-times-3-important-limits/>

Projekt: Software Engineering

Spezifikationsdokument

MietManager

Portfolio von Michael Bangas

Matrikelnummer: IU14132885

Kursbezeichnung: Projekt: Software Engineering (DLMCSPSE01_D)

Studiengang: M.Sc. Informatik

IU Internationale Hochschule

25. September 2026

Inhaltsverzeichnis

1	Datenmodell	1
1.1	Geschäftsobjekte	1
1.2	UML-Klassendiagramm	1
2	Geschäftsprozesse	2
2.1	Kernprozess: Nebenkostenabrechnung erstellen	2
3	Geschäftsregeln	3
4	Systemschnittstellen	4
5	Benutzerschnittstellen	4
5.1	Struktur	4
5.2	GUI-Mockups	5
5.3	Dialogflüsse und Eingabevalidierung	6
6	Fazit	6

Abbildungsverzeichnis

1.1	UML-Klassendiagramm	2
2.1	UML-Aktivitätsdiagramm: Nebenkostenabrechnung erstellen	3
5.1	Handskizzierte GUI-Mockups der zentralen Ansichten	5
5.2	Digitale GUI-Mockups der zentralen Ansichten	6

1 Datenmodell

Dieser Abschnitt beschreibt die zentralen Geschäftsobjekte von MietManager und ihre Beziehungen zueinander, zunächst textuell und anschließend als UML-Klassendiagramm.

1.1 Geschäftsobjekte

MietManager verwaltet als zentrales Geschäftsobjekt die Immobilie. Dabei handelt es sich um ein vermietetes Gebäude, identifiziert durch Bezeichnung und Adresse, das beliebig viele Mieteinheiten, Kostenpositionen und Nebenkostenabrechnungen besitzt. Eine Mieteinheit ist eine einzelne vermietbare Wohnung innerhalb einer Immobilie, deren Fläche und Zimmeranzahl als Grundlage für die Umlage von Kosten dienen. Ein Mieter ist eine natürliche Person mit Kontaktdaten, die einen oder mehrere Mietverträge abschließen kann. Ein Mietvertrag verknüpft einen Mieter mit einer Mieteinheit für einen Zeitraum und legt Kaltmiete, Nebenkostenvorauszahlung, Kautions- und Personenzahl fest.

Eine Kostenposition ist eine einzelne Betriebskostenart einer Immobilie für einen Erfassungszeitraum, inklusive des zu verwendenden Umlageschlüssels. Die Nebenkostenabrechnung ist das Ergebnis der Abrechnungsberechnung einer Immobilie für einen Zeitraum und besteht aus einer Abrechnungssposition je betroffenem Mietvertrag, dem Kostenanteil, geleisteter Vorauszahlung und Saldo enthält. Das Vermieterprofil bündelt die Stammdaten des Vermieters, wobei es ein eigenständiges Objekt ohne Beziehung zu den übrigen Geschäftsobjekten ist und als Briefkopf-Absender beim PDF-Export dient.

1.2 UML-Klassendiagramm

Abbildung 1.1 visualisiert die Geschäftsobjekte und ihre Beziehungen. Kompositionsbeziehungen (gefüllte Raute) bedeuten, dass das untergeordnete Objekt nicht ohne das übergeordnete existieren kann. Einfache Assoziationen (Linie) verknüpfen unabhängig existierende Objekte. Diese Geschäftsobjekte realisieren gemeinsam die im Anforderungsdokument als UML-Use-Case-Diagramm dargestellten Anwendungsfälle des Akteurs Vermieter. Welche der hier gezeigten Objekte dabei am Kernprozess „Nebenkostenabrechnung erstellen“ beteiligt sind, zeigt das UML-Aktivitätsdiagramm in Abschnitt 2.1.

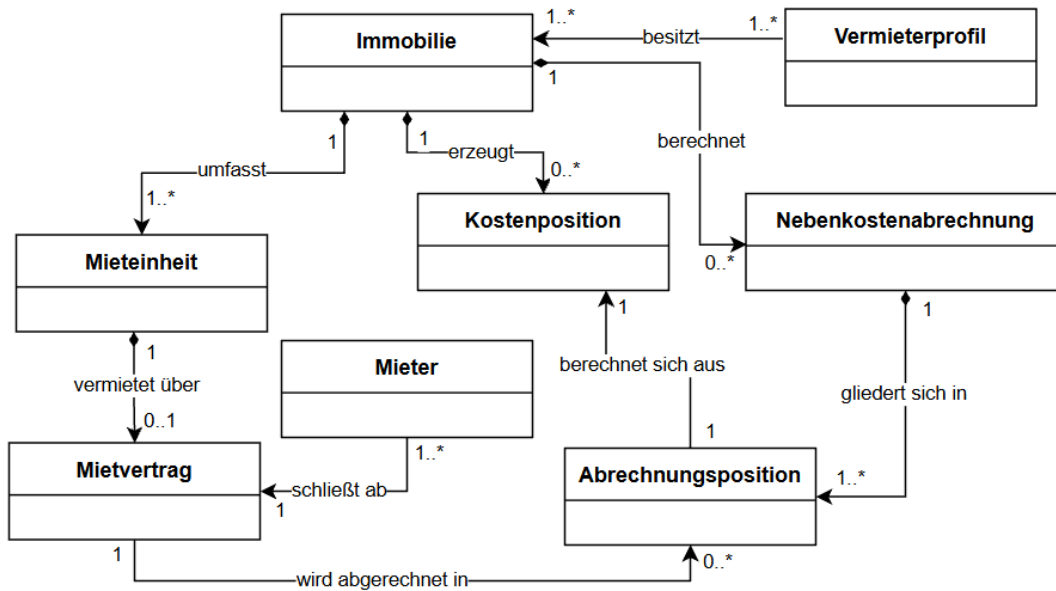


Abbildung 1.1: UML-Klassendiagramm

2 Geschäftsprozesse

Dieser Abschnitt beschreibt den zentralen Geschäftsprozess von MietManager textuell und visualisiert ihn anschließend als UML-Aktivitätsdiagramm.

2.1 Kernprozess: Nebenkostenabrechnung erstellen

Der zentrale Geschäftsprozess von MietManager berechnet die Nebenkostenabrechnung einer Immobilie für einen gewählten Zeitraum. Zunächst wählt der Vermieter eine Immobilie sowie Start- und Enddatum des Abrechnungszeitraums. Das System lädt daraufhin alle Kostenpositionen der Immobilie, die den Zeitraum überschneiden, und ermittelt alle im Zeitraum aktiven Mietverträge. Liegen keine Kostenpositionen oder keine aktiven Mietverträge vor, bricht das System mit einer Fehlermeldung ab. Andernfalls berechnet das System für jede Kostenposition je aktivem Mietvertrag den Anteil nach dem hinterlegten Umlageschlüssel und summiert die Anteile je Mietvertrag auf. Die Umlageschlüssel sind dabei Fläche, Personenzahl oder Anzahl Einheiten. Anschließend stellt das System den aufsummierten Kostenanteil der geleisteten Nebenkostenvorauszahlung gegenüber und berechnet den Saldo. Ist der Saldo positiv, wird er als Nachzahlung identifiziert, andernfalls als Rückerstattung. Das Ergebnis wird als Nebenkostenabrechnung mit einer Abrechnungsposition je Mietvertrag gespeichert, sodass der Vermieter die Abrechnung abschließend als personalisierten PDF-Brief je Mieter exportieren kann. Die Stammdaten des Vermieters werden dabei als Briefkopf-Absender verwendet und können im Profil gepflegt werden. Abbildung 2.1 visualisiert den beschriebenen Kernprozess.

Die dabei verarbeiteten Objekte Kostenposition, Mietvertrag, Nebenkostenabrechnung und Abrechnungsposition entsprechen den gleichnamigen Geschäftsobjekten aus dem UML-Klassendiagramm in Abbildung 1.1. Der Prozess selbst realisiert den im Anforderungsdokument als UML-Use-Case-Diagramm dargestellten Anwendungsfall „Nebenkostenabrechnung erstellen“ (FA-05).

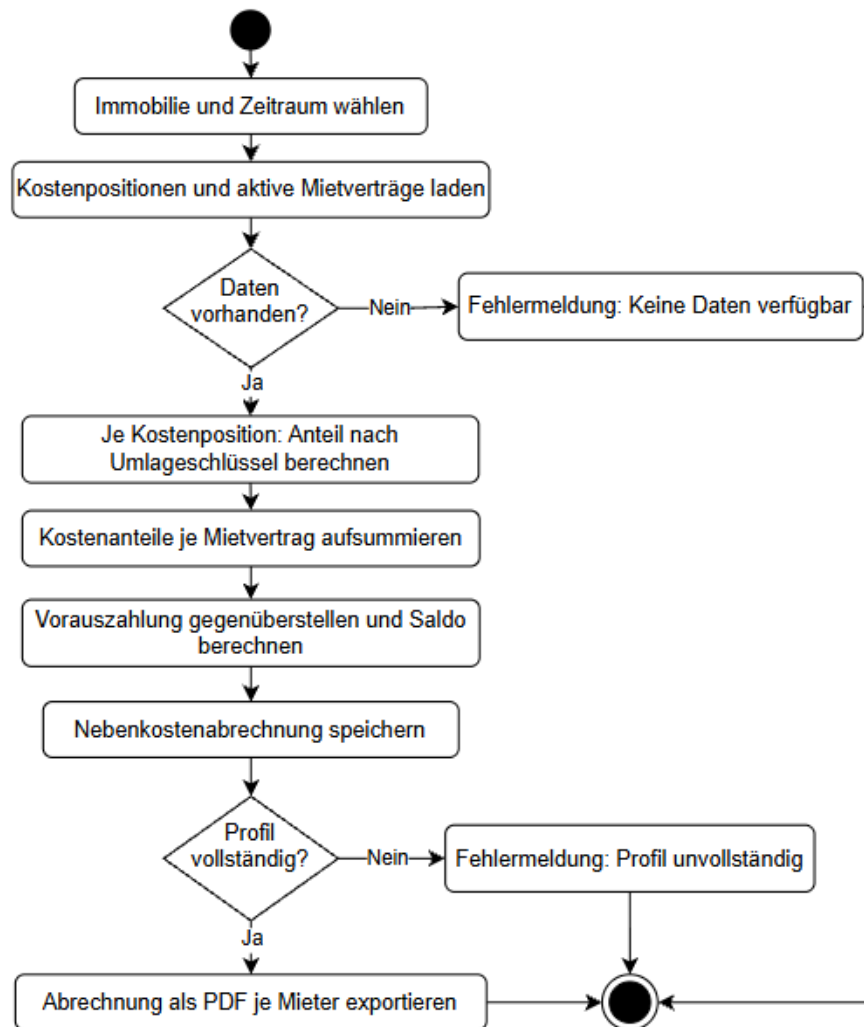


Abbildung 2.1: UML-Aktivitätsdiagramm: Nebenkostenabrechnung erstellen

3 Geschäftsregeln

Für Mietverträge gilt, dass das Startdatum vor dessen Enddatum liegen muss, sofern ein Enddatum gesetzt ist. Unbefristete Verträge haben kein Enddatum. Zudem darf eine Mieteinheit im selben Zeitraum nicht durch mehr als einen Mietvertrag belegt sein, sodass aktive Verträge derselben Mieteinheit sich zeitlich nicht überlappen dürfen.

Beim Löschen von Datensätzen greifen weitere Regeln, die verhindern sollen, dass ein gelöschter Datensatz referenzierende Datensätze verwaist zurücklässt oder eine bereits erstellte Nebenkosten-

abrechnung nachträglich ihre Grundlage verliert. Eine Immobilie lässt sich daher nur löschen, wenn keine ihrer Mieteinheiten einen bestehenden Mietvertrag hat. Aus demselben Grund kann eine Mieteinheit nur gelöscht werden, wenn kein Mietvertrag auf sie verweist, und ein Mieter nur, wenn ihm kein Mietvertrag mehr zugeordnet ist. Ein Mietvertrag wiederum lässt sich nur löschen, wenn ihm keine Abrechnungsposition aus einer bereits erstellten Nebenkostenabrechnung zugeordnet ist.

Für die Nebenkostenabrechnung gilt schließlich, dass der Start eines Abrechnungszeitraums vor dessen Ende liegen muss und eine Abrechnung nur erstellt werden kann, wenn für den Zeitraum mindestens eine Kostenposition und mindestens ein aktiver Mietvertrag der gewählten Immobilie vorliegen. Für den anschließenden PDF-Export müssen im Vermieterprofil mindestens Name, Straße, Postleitzahl und Ort hinterlegt sein.

4 Systemschnittstellen

MietManager ist eine lokale Desktop-Anwendung ohne Anbindung an externe Systeme oder Netzwerkdienste. Die einzige technische Schnittstelle nach außen ist der Dateisystemzugriff. Zur Datenehaltung dient eine lokale SQLite-Datenbankdatei im Benutzerprofil des Betriebssystems, und beim PDF-Export erzeugt die Anwendung PDF-Dateien im vom Benutzer gewählten Zielverzeichnis des Dateisystems.

5 Benutzerschnittstellen

Dieser Abschnitt beschreibt die Struktur der grafischen Benutzeroberfläche von MietManager sowie die Dialogflüsse und die Eingabevalidierung bei der Datenerfassung.

5.1 Struktur

Die Anwendung besteht aus einem Hauptfenster mit tabellarischer Navigation und einem Profil-Popup. Das Dashboard zeigt eine Kennzahlen-Übersicht für Immobilien, Mieteinheiten, Leerstand, aktive Verträge und Mieteinnahmen als Karten. Im Tab Immobilien werden die Immobilien mit ihren Mieteinheiten in einer Baumansicht dargestellt, inklusive Anlegen, Bearbeiten und Löschen. Die Tabs Mieter, Mietverträge und Kostenpositionen bieten jeweils eine tabellarische Verwaltung der entsprechenden Datensätze. Der Tab Nebenkostenabrechnung enthält ein Formular zur Auswahl von Immobilie und Zeitraum, eine tabellarische Ergebnisanzeige sowie den PDF-Export. Über den Profil-Button oben rechts öffnet sich das Profil-Popup mit dem Formular zur Pflege der Vermieter-Stammdaten.

5.2 GUI-Mockups

Abbildung 5.1 zeigt handskizzierte und Abbildung 5.2 die daraus mit dem spezialisierten Design-Werkzeug Figma abgeleiteten digitalen Mockups der zentralen Ansichten Dashboard, tabellarischer Daten-Tab, stellvertretend für Immobilien, Mieter, Mietverträge und Kostenpositionen, Tab Nebenkostenabrechnung sowie das Profil-Popup.

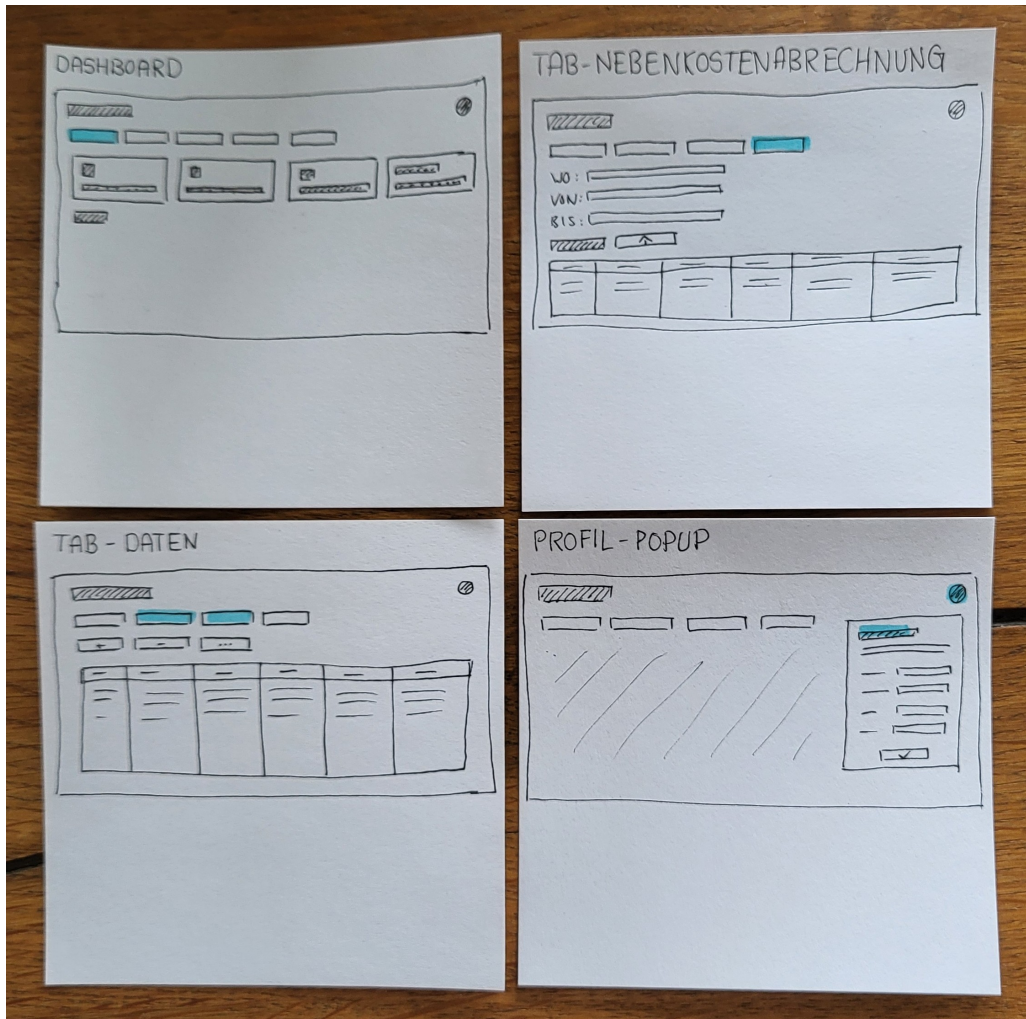
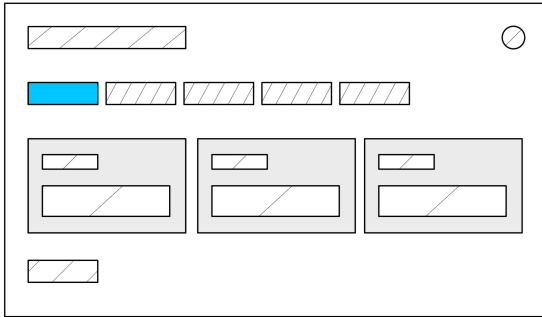


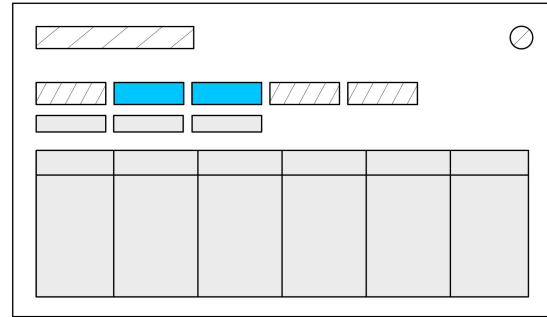
Abbildung 5.1: Handskizzierte GUI-Mockups der zentralen Ansichten

DASHBOARD



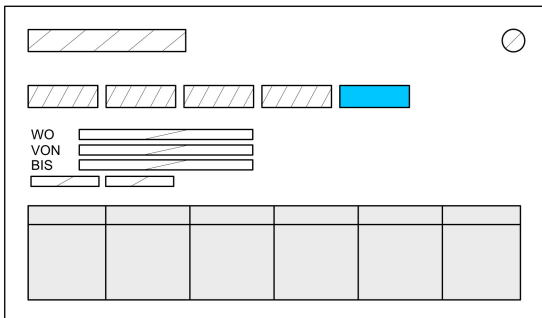
(a) Dashboard

TAB - DATEN



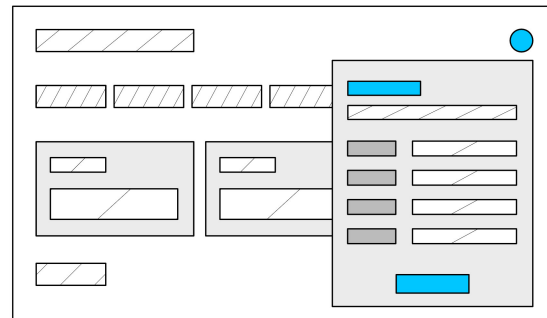
(b) Tab-Daten

TAB - NEBENKOSTENABRECHUNG



(c) Tab Nebenkostenabrechnung

PROFIL - POPUP



(d) Profil-Popup

Abbildung 5.2: Digitale GUI-Mockups der zentralen Ansichten

5.3 Dialogflüsse und Eingabevalidierung

Neue oder zu bearbeitende Datensätze werden über Dialoge mit Formularfeldern erfasst, die mit „OK“/„Abbrechen“ bestätigt bzw. verworfen werden. Pflichtfelder werden beim Bestätigen geprüft. Bei ungültiger Eingabe erscheint eine Fehlermeldung mit konkretem Hinweis, der Dialog bleibt geöffnet. Für den Mietvertrags-Dialog wird zusätzlich die Geschäftsregel zur zeitlichen Überschneidung je Mieteinheit geprüft. Löscht-Aktionen erfordern eine Sicherheitsabfrage und werden verweigert, wenn die zugehörige Geschäftsregel eine Löschung ausschließt.

6 Fazit

Dieses Dokument hat das Datenmodell mit seinen acht Geschäftsobjekten und deren Beziehungen als UML-Klassendiagramm spezifiziert, den Kernprozess „Nebenkostenabrechnung erstellen“ als UML-Aktivitätsdiagramm im Detail beschrieben und die zentralen Geschäftsregeln zu Vertragszeiträumen sowie zum Löschen referenzierter Datensätze festgelegt. Zusammen mit den System-schnittstellen und den Benutzerschnittstellen inklusive GUI-Mockups und Eingabevalidierung ergibt

sich damit eine vollständige fachliche Spezifikation, auf deren Grundlage das Architekturdokument die technische Umsetzung in der Erarbeitungsphase beschreibt.

Projekt: Software Engineering

Architekturdokument

MietManager

Portfolio von Michael Bangas

Matrikelnummer: IU14132885

Kursbezeichnung: Projekt: Software Engineering (DLMCSPSE01_D)

Studiengang: M.Sc. Informatik

IU Internationale Hochschule

25. September 2026

Inhaltsverzeichnis

1	Technologieübersicht	1
1.1	Eingesetzte Technologien	1
1.2	Betrachtete Alternativen	1
2	Architekturübersicht	2
3	Struktur	3
4	Verhalten	4
5	Fazit	5

Abbildungsverzeichnis

2.1	Schichtenarchitektur von MietManager	3
3.1	UML-Klassendiagramm der Geschäftsobjekte von MietManager.	4
4.1	UML-Sequenzdiagramm zum Prozess der Erstellung und Export der Nebenkostenabrechnung.	5

1 Technologieübersicht

Dieser Abschnitt beschreibt zunächst die eingesetzten Technologien und Werkzeuge von MietManager und stellt anschließend die jeweils betrachteten Alternativen sowie die Gründe für ihre Ablehnung dar.

1.1 Eingesetzte Technologien

MietManager ist in Python 3.14 implementiert, das durch seine breite Standardbibliothek, die kurze Entwicklungszeit durch dynamische Typisierung und die gute Verfügbarkeit ausgereifter Pakete für alle benötigten Teilbereiche wie GUI, PDF-Erzeugung und Tests für ein zeitlich auf vier Wochen begrenztes Portfolioprojekt geeignet ist. Für die grafische Oberfläche kommt PyQt6 zum Einsatz, das unter Windows ein natives Erscheinungsbild, ein umfangreiches Widget-Set sowie mit Qt Style Sheets eine deklarative Möglichkeit zur einheitlichen Gestaltung der Oberfläche bietet. Die Datenhaltung erfolgt über SQLAlchemy 2.0, das die Geschäftsobjekte auf Tabellen einer lokalen SQLite-Datenbank abbildet. SQLite kommt dabei dateibasiert ohne separate Serverinstallation aus, was zur Zielgruppe einer lokalen Single-User-Desktop-Anwendung ohne Netzwerkanbindung passt. Für den PDF-Export der Nebenkostenabrechnung wird ReportLab verwendet, das die Geschäftsbriefe direkt programmatisch erzeugt. Die Geschäftsregeln und die Berechnungslogik werden mit pytest getestet, das flache Testfunktionen statt Testklassen ermöglicht und die Service-Schicht unabhängig von Qt und Datenbank rein in Python prüfbar macht. Mit coverage.py wird zusätzlich gemessen, wie vollständig diese Tests die Service-Schicht tatsächlich abdecken. Für die Auslieferung an Endanwender ohne installierten Python-Interpreter ist PyInstaller vorgesehen, das die Anwendung in eine eigenständige Windows-.exe überführt und damit die im Anforderungsdokument definierte Portabilitätsanforderung erfüllt.

1.2 Betrachtete Alternativen

Als Alternative zu Python kämen die im Rahmen dieses Kurses ebenfalls zugelassenen Sprachen Java, C++ oder C# infrage. Diese hätten zwar eine strengere statische Typisierung geboten, doch der damit verbundene höhere Boilerplate-Aufwand erschien angesichts der knappen Projektdauer nicht gerechtfertigt, weshalb die Wahl auf Python fiel.

Für die grafische Oberfläche stand PyQt6 der in der Python-Standardbibliothek enthaltenen Alternative Tkinter gegenüber. Tkinter käme ohne zusätzliche Abhängigkeit aus, produziert unter Windows jedoch ein wenig natives Erscheinungsbild und bietet kein deklaratives Stylesheet-Konzept für eine einheitliche Gestaltung mehrerer Tabs und Dialoge, weshalb die Wahl auf PyQt6 fiel.

Im Vergleich zu einer Client-Server-Datenbank wie PostgreSQL oder MySQL vermeidet die dateiba-

sierte SQLite-Datenbank den zusätzlichen Installations- und Wartungsaufwand eines Datenbankservers, den PostgreSQL oder MySQL erfordert hätten, ohne dass die Zielgruppe der lokalen Single-User-Anwendung auf dessen Fähigkeiten angewiesen wäre.

Für den PDF-Export wurde ReportLab HTML-basierten Alternativen wie WeasyPrint vorgezogen, da es die Geschäftsbriefe direkt programmatisch erzeugt, statt den Umweg über eine zusätzliche Templating-Schicht zu gehen. Für die Tests wurde pytest dem in der Standardbibliothek enthaltenen unittest vorgezogen, da pytest flache Testfunktionen statt Testklassen ermöglicht.

2 Architekturübersicht

MietManager folgt einer klassischen Drei-Schichten-Architektur, die Benutzeroberfläche, Geschäftslogik und Datenhaltung strikt voneinander trennt. Alternative Architekturmuster wie eine Microservices-Architektur oder eine Client-Server-Architektur mit eigenständigem Backend-Dienst wurden verworfen, da MietManager als lokale Single-User-Desktop-Anwendung ohne Netzwerkanbindung betrieben wird und die mit einer Aufteilung in unabhängig deploybare Dienste verbundene zusätzliche Komplexität, etwa durch Interprozesskommunikation oder Serviceorchestrierung, hier keinen Mehrwert böte. Eine Schichtenarchitektur erlaubt stattdessen bereits innerhalb eines einzelnen Prozesses die für die im Testdokument beschriebene isolierte Testbarkeit der Geschäftslogik nötige Trennung, ohne die Betriebskomplexität eines verteilten Systems einzuführen.

Die UI-Schicht im Paket `mietmanager.ui` besteht aus dem Hauptfenster, den tabellarischen Tabs sowie den Erfassungsdialogen. Sie reagiert auf Benutzerinteraktion, ruft die darunterliegende Serviceschicht auf und stellt deren Ergebnisse bzw. Fehlermeldungen dar, enthält jedoch selbst keine Geschäftslogik. Die Serviceschicht im Paket `mietmanager.services` kapselt die Geschäftsregeln, die Berechnung der Nebenkostenabrechnung sowie den PDF-Export und ist vollständig unabhängig von PyQt6 und damit ohne laufende Anwendung testbar. Die Datenzugriffsschicht in den Paketen `mietmanager.data` und `mietmanager.models` kapselt den Zugriff auf die SQLite-Datenbank über SQLAlchemy-Modelle, Engine und Session. Abbildung 2.1 visualisiert die drei Schichten und ihre Abhängigkeitsrichtung. Aufrufe verlaufen ausschließlich von der UI- über die Service- zur Datenzugriffsschicht, nie umgekehrt, sodass die im Spezifikationsdokument definierten Geschäftsregeln unabhängig von der Oberfläche gelten.

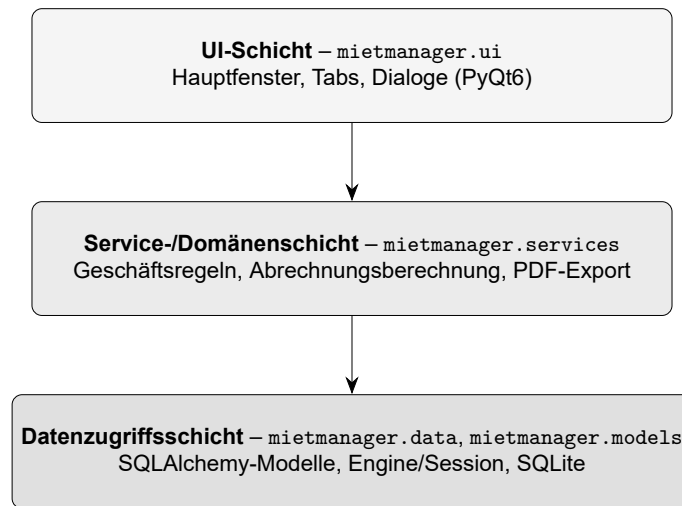


Abbildung 2.1: Schichtenarchitektur von MietManager

3 Struktur

Die Struktur von MietManager wird über dieselben acht Geschäftsobjekte beschrieben, die bereits im Spezifikationsdokument als UML-Klassendiagramm eingeführt wurden. Es handelt sich um Immobilie, Mieteinheit, Mieter, Mietvertrag, Kostenposition, Nebenkostenabrechnung, Abrechnungsposition und Vermieterprofil. Während das Klassendiagramm im Spezifikationsdokument den fachlichen Zusammenhang der Geschäftsobjekte im Vordergrund stellt, zeigt Abbildung 3.1 dieselben Klassen zusätzlich mit ihren wichtigsten Attributen und Methoden, wie sie in `models/entities.py` als SQLAlchemy-Modellklassen implementiert sind. Über die reine Datenhaltung hinaus besitzt von den acht Klassen nur `Vermieterprofil` eigenes Verhalten. Die Methode `is_complete()` prüft, ob Name, Straße, Postleitzahl und Ort gesetzt sind, und wird von der Service-Schicht vor dem PDF-Export aufgerufen. Dass die Geschäftsobjekte insgesamt so wenige Methoden besitzen, ist eine bewusste Folge der in Abschnitt 2 beschriebenen Schichtenarchitektur. Die Modellklassen in `models/entities.py` sind reine SQLAlchemy-Datenklassen ohne Geschäftslogik, während sämtliche Berechnungen und Regeln in der Service-Schicht als eigenständige Funktionen liegen, die Modellobjekte lediglich als Parameter entgegennehmen und mit dem Ergebnis befüllen. So besitzen beispielsweise weder `Nebenkostenabrechnung` noch `Abrechnungsposition` eine Berechnungsmethode. Die komplette Berechnungslogik der Nebenkostenabrechnung steckt stattdessen in der Funktion `erstelle_abrechnung(session, immobilie, start, ende)` aus `services/abrechnung.py`, die in Abbildung 4.1 als Aufruf von `AbrechnungTab` an die Service-Schicht dargestellt ist. Diese Trennung hält die Modellklassen schlank und stellt sicher, dass Geschäftslogik wie in Abschnitt 2 gefordert unabhängig von Qt und Datenbank testbar bleibt.

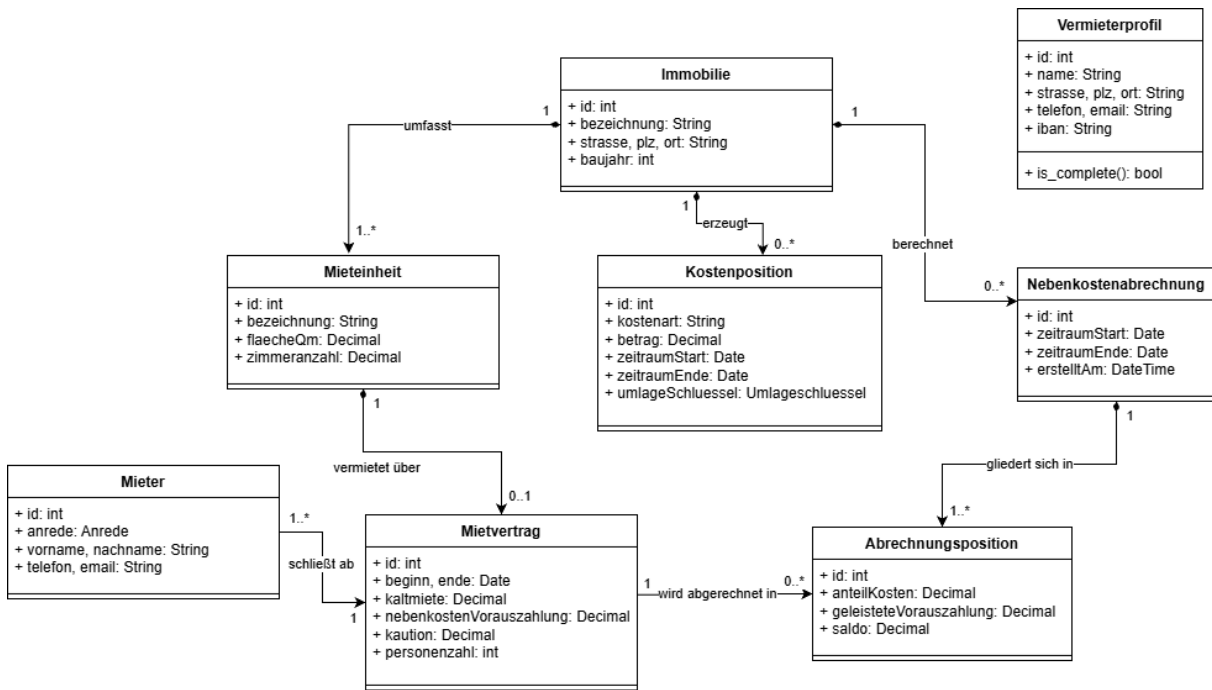


Abbildung 3.1: UML-Klassendiagramm der Geschäftsobjekte von MietManager.

4 Verhalten

Als zentraler Systemablauf wird der Kernprozess „Nebenkostenabrechnung erstellen und als PDF exportieren“ betrachtet, der bereits im Spezifikationsdokument fachlich als UML-Aktivitätsdiagramm beschrieben ist und in Abbildung 4.1 technisch als UML-Sequenzdiagramm mit den tatsächlichen Funktionsaufrufen über die Schichten hinweg dargestellt wird. Jeder durchgängige Hin-Pfeil entspricht dabei einem konkreten Aufruf aus dem Quellcode. Besitzt der Aufruf einen Rückgabewert, zeigt der zugehörige gestrichelte Rückgabe-Pfeil die Zuweisung an die im Code tatsächlich verwendete Variable, z. B. `dateien = exportiere_abrechnung(...)`. Die schmalen Balken auf den Lebenslinien markieren die Aktivierungsdauer. Die drei Service-Module `abrechnung.py`, `regeln.py` und `pdf_export.py` sind dabei als eigene Lebenslinien dargestellt, statt sie wie in Abschnitt 2 zur Serviceschicht zusammenzufassen, damit erkennbar bleibt, welcher Aufruf tatsächlich in welchem Modul landet. Der Vermieter löst über den Klick auf „Abrechnung erstellen“ die gleichnamige Methode auf `AbrechnungTab` aus, die ihrerseits die Berechnungsfunktion `erstelle_abrechnung` aus `services.abrechnung` aufruft. Diese persistiert das Ergebnis über die SQLAlchemy-Session und gibt das `Nebenkostenabrechnung`-Objekt zurück. Beim anschließenden PDF-Export prüft `AbrechnungTab` zunächst per `pruefe_profil_vollstaendig` aus `services.regeln`, ob das `Vermieterprofil` vollständig ist. Anschließend wird `exportiere_abrechnung` aus `services.pdf_export` aufgerufen und je Mietvertrag ein PDF-Brief in das gewählte Zielverzeichnis geschrieben.

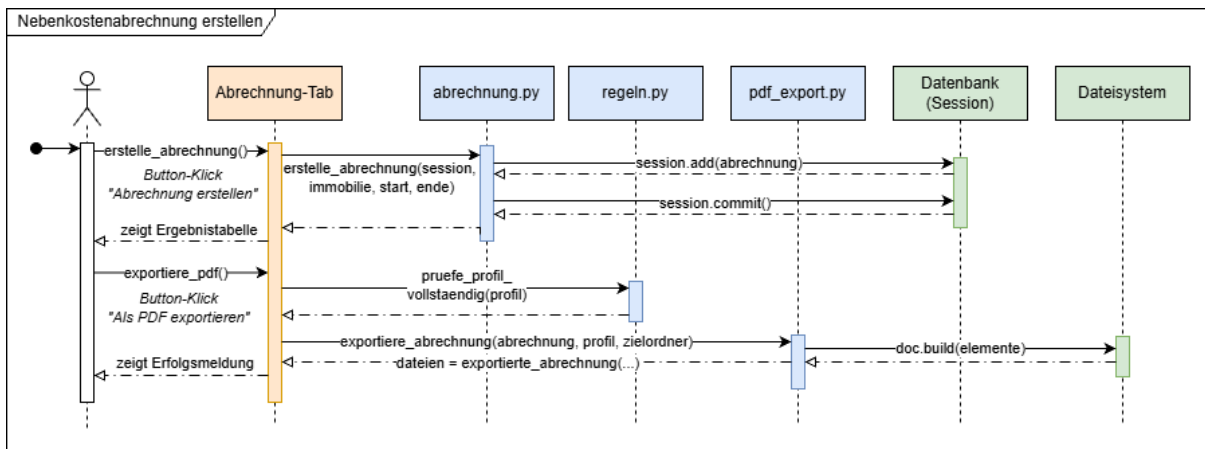


Abbildung 4.1: UML-Sequenzdiagramm zum Prozess der Erstellung und Export der Nebenkostenabrechnung.

5 Fazit

Dieses Dokument hat die technische Umsetzung der im Spezifikationsdokument fachlich beschriebenen Anforderungen dargestellt. Dazu zählen die eingesetzten Technologien, die Drei-Schichten-Architektur aus UI-, Service- und Datenzugriffsschicht, die acht Geschäftsobjekte als schlanke SQLAlchemy-Modellklassen sowie der Kernprozess „Nebenkostenabrechnung erstellen und als PDF exportieren“ als UML-Sequenzdiagramm mit den tatsächlichen Funktionsaufrufen über die Schichten hinweg. Auf dieser technischen Grundlage wird die Anwendung implementiert und anschließend im Testdokument der Finalisierungsphase gegen Unit- und Systemtests verifiziert.

Projekt: Software Engineering

Testdokument

MietManager

Portfolio von Michael Bangas

Matrikelnummer: IU14132885

Kursbezeichnung: Projekt: Software Engineering (DLMCSPSE01_D)

Studiengang: M.Sc. Informatik

IU Internationale Hochschule

25. September 2026

Inhaltsverzeichnis

1	Teststrategie	1
2	Unit-Tests	1
3	Testprotokoll	3
4	Performance-Test	6
5	Fazit	7

Tabellenverzeichnis

2.1	Automatisierte Unit-Tests	1
3.1	Testprotokoll der manuellen Systemtests	3
4.1	Ergebnis des Performance-Benchmarks bei maximaler dokumentierter Skalierung . .	7

1 Teststrategie

Die Qualitätssicherung von MietManager gliedert sich in drei Teststufen, die jeweils unterschiedliche Aspekte der Anwendung absichern. Automatisierte Unit-Tests prüfen mit `pytest` die gesamte Service-Schicht direkt auf Funktionsebene, unabhängig von grafischer Oberfläche. Sie liegen im Verzeichnis `tests/` und lassen sich jederzeit reproduzierbar mit dem Befehl `pytest` ausführen. Dass diese Schicht überhaupt unabhängig testbar ist, ist eine unmittelbare Folge der in Anforderungs- und Architekturdokument beschriebenen Drei-Schichten-Architektur, in der Geschäftslogik bewusst aus der UI-Schicht herausgelöst ist. Mit dem Werkzeug `coverage.py` gemessen erreicht die Service-Schicht dadurch 98–100 % Testabdeckung je Modul (`regeln.py` und `pdf_export.py` 100 %, `abrechnung.py` 98 %). Ergänzend werden Systemtests als manuelle End-zu-Ende-Durchläufe über die grafische Oberfläche durchgeführt, die die Interaktion aller Schichten inklusive Datenbank und PDF-Export abdecken und anhand des Testprotokolls in Abschnitt 3 dokumentiert werden. Die UI- und Datenzugriffsschicht selbst bleiben bewusst ohne automatisierte Tests, da sich Qt-Oberflächen ohne unverhältnismäßigen Mehraufwand kaum sinnvoll unit-testen lassen. Auf dedizierte Integrationstests unterhalb der Systemtestebene wird verzichtet, da die Service-Schicht in den Unit-Tests bereits mit einer echten SQLite-Datenbank statt gemockter Objekte geprüft wird und der verbleibende Integrationsaspekt, das Zusammenspiel mit der tatsächlichen SQLite-Datenbank und der Oberfläche, bereits durch die Systemtests abgedeckt ist. Ergänzend prüft ein separates Benchmark-Skript in Abschnitt 4 die im Anforderungsdokument quantifizierten nichtfunktionalen Anforderungen an Performance und Skalierbarkeit, die von den funktionalen Testebenen nicht abgedeckt werden.

2 Unit-Tests

Tabelle 2.1 listet alle 42 automatisierten Unit-Tests aus dem Verzeichnis `tests/` auf. `test_regeln.py` prüft sämtliche Funktionen aus `services/regeln.py` gegen einen rein im Speicher aufgebauten Objektgraphen. `test_abrechnung.py` und `test_pdf_export.py` prüfen die Berechnungslogik bzw. den PDF-Export aus `services/abrechnung.py` und `services/pdf_export.py` gegen eine echte, In-Memory aufgebaute SQLite-Datenbank, da diese Funktionen selbst Datensätze speichern bzw. lesen. `test_setup.py` verifiziert das Testsetup selbst. Zum Zeitpunkt der Erstellung dieses Dokuments bestehen alle 42 Tests reproduzierbar unter Python 3.14 (Befehl `pytest tests/ -v`).

Tabelle 2.1: Automatisierte Unit-Tests

Datei	Testfall	Ergebnis
<code>test_regeln.py</code>	<code>test_gueltiger_zeitraum_wird_akzeptiert</code>	bestanden

Tabelle 2.1 – Fortsetzung

Datei	Testfall	Ergebnis
	test_unbefristeter_vertrag_ohne_ende_wird_akzeptiert	bestanden
	test_ende_vor_beginn_wird_abgelehnt	bestanden
	test_ende_gleich_beginn_wird_abgelehnt	bestanden
	test_kein_konflikt_bei_nicht_ueberlappenden_zeitraeumen	bestanden
	test_konflikt_bei_ueberlappenden_zeitraeumen	bestanden
	test_eigener_vertrag_wird_beim_bearbeiten_ausgenommen	bestanden
	test_immobilie_ohne_vertraege_ist_loeschbar	bestanden
	test_immobilie_mit_vertrag_ist_nicht_loeschbar	bestanden
	test_mieteinheit_ohne_vertrag_ist_loeschbar	bestanden
	test_mieteinheit_mit_vertrag_ist_nicht_loeschbar	bestanden
	test_mieter_ohne_vertrag_ist_loeschbar	bestanden
	test_mieter_mit_vertrag_ist_nicht_loeschbar	bestanden
	test_mietvertrag_ohne_abrechnungsposition_ist_loeschbar	bestanden
	test_mietvertrag_mit_abrechnungsposition_ist_nicht_loeschbar	bestanden
	test_vollstaendiges_profil_wird_akzeptiert	bestanden
	test_unvollstaendiges_profil_wird_abgelehnt	bestanden
test_abrechnung.py	test_start_nach_ende_wird_abgelehnt	bestanden
	test_ohne_kostenpositionen_wird_abgelehnt	bestanden
	test_ohne_aktive_vertraege_wird_abgelehnt	bestanden
	test_gesamtgewicht_null_wird_abgelehnt	bestanden
	test_kostenposition_ausserhalb_des_zeitraums_wird_ignoriert	bestanden
	test_verteilung_nach_flaeche	bestanden
	test_verteilung_nach_personenzahl	bestanden
	test_verteilung_nach_einheiten_ist_gleichmaessig	bestanden
	test_saldo_positiv_bedeutet_nachzahlung	bestanden
	test_beendeter_vertrag_ausserhalb_zeitraum_wird_nicht_beruecksichtigt	bestanden
	test_persistiert_abrechnung_in_datenbank	bestanden
	test_get_positionsdetails_liefert_kostenart_und_anteil_je_kostenposition	bestanden

Tabelle 2.1 – Fortsetzung

Datei	Testfall	Ergebnis
	test_lade_abrechnungen_liefert_leere_liste_ohne_abrechnungen	bestanden
	test_lade_abrechnungen_liefert_nur_abrechnungen_der_gewaehlten_immobilie	bestanden
	test_lade_abrechnungen_sortiert_neuesten_zeitraum_zuerst	bestanden
test_pdf_export.py	test_anredetext_herr	bestanden
	test_anredetext_frau	bestanden
	test_anredetext_keine_angabe	bestanden
	test_dateiname_enthaelt_jahr_und_mieternamen	bestanden
	test_dateiname_ersetzt_dateisystem_unsichere_zeichen	bestanden
	test_exportiere_abrechnung_lehnt_unvollstaendiges_profil_ab	bestanden
	test_exportiere_abrechnung_erzeugt_eine_pdf_datei_je_mietvertrag	bestanden
	test_exportiere_abrechnung_bei_nachzahlung_mit_iban	bestanden
	test_exportiere_abrechnung_bei_rueckerstattung	bestanden
test_setup.py	test_setup_ok	bestanden

3 Testprotokoll

Tabelle 3.1 dokumentiert die manuellen Systemtests, die die Bedienung über die grafische Oberfläche Ende-zu-Ende abdecken, einschließlich Fällen, die keine reine Geschäftsregel im Sinne von Abschnitt 2 sind, sondern die Dialogvalidierung, den Datenbankzugriff oder den PDF-Export betreffen. Zu jedem Anlegen-Dialog ist dabei sowohl der positive Fall als auch mindestens ein negativer Fall aufgeführt. Ergänzend sind Bearbeiten-Fälle sowie zu jeder Löschregel aus dem Spezifikationsdokument sowohl der positive als auch der negative Fall enthalten.

Tabelle 3.1: Testprotokoll der manuellen Systemtests

ID	Kurzbeschreibung	Vorbedingungen	Eingabedaten / Aktionen	Erwartetes Ergebnis	Tatsächl. Ergebnis
Immobilie					
TF-01	Immobilie mit gültigen Daten anlegen	Anwendung gestartet, Tab Immobilien	Dialog „Neue Immobilie“ öffnen, Bezeichnung und Straße ausfüllen, „OK“	Immobilie erscheint in der Baumansicht	positiv

Tabelle 3.1 – Fortsetzung

ID	Kurzbeschreibung	Vorbedingungen	Eingabedaten / Aktionen	Erwartetes Ergebnis	Tatsächl. Ergebnis
TF-02	Immobilie ohne Pflichtfelder anlegen	Anwendung gestartet, Tab Immobilien	Dialog „Neue Immobilie“ öffnen, Bezeichnung und Straße leer lassen, „OK“	Warnung „Bezeichnung und Straße sind Pflichtfelder“, Dialog bleibt offen	positiv
TF-03	Immobilie bearbeiten	Mindestens eine Immobilie vorhanden	Immobilie auswählen, „Bearbeiten...“, Bezeichnung ändern, „OK“	Geänderte Bezeichnung erscheint in der Baumansicht	positiv
TF-04	Immobilie ohne bestehende Mietverträge löschen	Immobilie ohne Mietvertrag mit aktivem Mietvertrag	Immobilie auswählen, „Löschen“, Sicherheitsabfrage bestätigen	Immobilie wird aus der Baumansicht entfernt	positiv
TF-05	Löschen einer Immobilie mit bestehendem Mietvertrag verweigern	Immobilie besitzt Mietvertrag mit aktivem Mietvertrag	Immobilie auswählen, „Löschen“	Warnung „Löschen nicht möglich“, Immobilie bleibt bestehen	positiv
Mieteinheit					
TF-06	Mieteinheit zu Immobilie anlegen	Mindestens eine Immobilie vorhanden	Immobilie auswählen, „Neue Mieteinheit“, Bezeichnung/Fläche/Zimmer eingeben, speichern	Mieteinheit erscheint als Kindknoten der Immobilie in der Baumansicht	positiv
TF-07	Mieteinheit bearbeiten	Mindestens eine Mieteinheit vorhanden	Mieteinheit auswählen, „Bearbeiten...“, Fläche ändern, „OK“	Geänderte Fläche erscheint in der Baumansicht	positiv
TF-08	Mieteinheit mit bestehendem Mietvertrag löschen verweigern	Mieteinheit hat einen aktiven Mietvertrag	Mieteinheit auswählen, „Löschen“	Warnung „Löschen nicht möglich“, Mieteinheit bleibt bestehen	positiv
Mieter					
TF-09	Mieter mit gültigen Daten anlegen	Anwendung gestartet, Tab Mieter	Dialog „Neuer Mieter“ öffnen, Vor- und Nachname ausfüllen, „OK“	Mieter erscheint in der Übersicht	positiv
TF-10	Mieter ohne Pflichtfelder anlegen	Anwendung gestartet, Tab Mieter	Dialog „Neuer Mieter“ öffnen, Vor-/Nachname leer lassen, „OK“	Warnung „Vor- und Nachname sind Pflichtfelder“, Dialog bleibt offen	positiv
TF-11	Mieter bearbeiten	Mindestens ein Mieter vorhanden	Mieter auswählen, „Bearbeiten...“, Telefonnummer ändern, „OK“	Geänderte Telefonnummer erscheint in der Übersicht	positiv
TF-12	Mieter ohne Mietvertrag löschen	Mieter ohne zugeordneten Mietvertrag vorhanden	Mieter auswählen, „Löschen“, Sicherheitsabfrage bestätigen	Mieter wird aus der Übersicht entfernt	positiv
TF-13	Mieter mit Mietvertrag löschen verweigern	Mieter hat mindestens einen Mietvertrag	Mieter auswählen, „Löschen“	Warnung „Löschen nicht möglich“, keine Sicherheitsabfrage, Mieter bleibt bestehen	positiv

Tabelle 3.1 – Fortsetzung

ID	Kurzbeschreibung	Vorbedingungen	Eingabedaten / Aktionen	Erwartetes Ergebnis	Tatsächl. Ergebnis
Mietvertrag					
TF-14	Mietvertrag mit gültigen Daten anlegen	Mieteinheit und Mieter vorhanden, Mieteinheit noch nicht vermietet	Dialog „Neuer Mietvertrag“ öffnen, Mieteinheit, Mieter, Beginn, Kaltmiete, NK-Vorauszahlung, Kaution und Personenzahl ausfüllen, „OK“	Mietvertrag erscheint in der Übersicht	positiv
TF-15	Mietvertrag mit Enddatum vor Startdatum ablehnen	Mieteinheit und Mieter vorhanden	„läuft unbefristet“ deaktivieren, Enddatum vor das Beginn-Datum setzen, „OK“	Warnung „Das Startdatum muss vor dem Enddatum liegen“, Dialog bleibt offen	positiv
TF-16	Mietvertrag mit überlappendem Zeitraum ablehnen	Mieteinheit hat bereits einen aktiven Mietvertrag	Neuen Mietvertrag für dieselbe Mieteinheit mit überschneidendem Zeitraum anlegen	Fehlermeldung mit Namen des bestehenden Mieters, Vertrag wird nicht gespeichert	positiv
TF-17	Mietvertrag bearbeiten	Mindestens ein Mietvertrag vorhanden	Mietvertrag auswählen, „Bearbeiten...“, Kaltmiete ändern, „OK“	Geänderte Kaltmiete scheint in der Übersicht	positiv
TF-18	Mietvertrag ohne Abrechnung löschen	Mietvertrag ohne zugeordnete Abrechnungssposition	Mietvertrag auswählen, „Löschen“, Sicherheitsabfrage bestätigen	Mietvertrag wird aus der Übersicht entfernt	positiv
TF-19	Löschen eines bereits abgerechneten Mietvertrags verweigern	Für den Mietvertrag wurde bereits eine Nebenkostenabrechnung erstellt	Mietvertrag auswählen, „Löschen“	Warnung „Löschen nicht möglich“, Mietvertrag bleibt bestehen	positiv
Kostenposition					
TF-20	Kostenposition mit gültigen Daten anlegen	Mindestens eine Immobilie vorhanden	Dialog „Neue Kostenposition“ öffnen, Immobilie, Kostenart, Betrag, Zeitraum und Umlageschlüssel ausfüllen, „OK“	Kostenposition erscheint in der Übersicht	positiv
TF-21	Kostenposition mit ungültigem Zeitraum ablehnen	Mindestens eine Immobilie vorhanden	Kostenposition mit Startdatum nach Enddatum anlegen, „OK“	Warnung „Das Startdatum muss vor dem Enddatum liegen“, Dialog bleibt offen	positiv
Nebenkostenabrechnung und PDF-Export					
TF-22	Abrechnung ohne Kostenpositionen ablehnen	Immobilie ohne erfasste Kostenpositionen im gewählten Zeitraum	Immobilie und Zeitraum wählen, „Abrechnung erstellen“	Warnung „Für den gewählten Zeitraum sind keine Kostenpositionen erfasst“	positiv

Tabelle 3.1 – Fortsetzung

ID	Kurzbeschreibung	Vorbedingungen	Eingabedaten / Aktionen	Erwartetes Ergebnis	Tatsächl. Ergebnis
TF-23	Nebenkostenabrechnung erfolgreich erstellen	Immobilie mit Kostenpositionen und mindestens einem aktiven Mietvertrag im Zeitraum	Immobilie und Zeitraum wählen, „Abrechnung erstellen“	Ergebnistabelle zeigt je Mietvertrag Kostenanteil, Vorauszahlung und Saldo. Summe der Kostenanteile entspricht der Summe der Kostenpositionen	positiv
TF-24	PDF-Export bei unvollständigem Vermieterprofil ablehnen	Erstellte Abrechnung liegt vor, Vermieterprofil ohne Name/Straße/PLZ/Ort	„Als PDF exportieren...“ klicken	Warnung „Vermieter-Profil unvollständig“, kein Dateiauswahldialog erscheint	positiv
TF-25	PDF-Export erfolgreich	Erstellte Abrechnung liegt vor, Vermieterprofil vollständig ausgefüllt	„Als PDF exportieren...“ klicken, Zielordner wählen	Für jeden Mietvertrag der Abrechnung liegt eine PDF-Datei mit korrektem Briefkopf im Zielordner	positiv
TF-26	Abrechnung im Verlauf löschen	Für die gewählte Immobilie liegt mindestens eine bereits erstellte Abrechnung im Verlauf vor	Abrechnung im Verlauf auswählen, „Abrechnung löschen“, Sicherheitsabfrage bestätigen	Abrechnung verschwindet aus dem Verlauf	positiv
Vermieterprofil und Dashboard					
TF-27	Vermieterprofil pflegen	Anwendung gestartet	Profil-Button oben rechts öffnen, Name, Straße, PLZ und Ort ausfüllen, „Speichern“	Popup schließt ohne Fehlermeldung, Daten sind nach erneutem Öffnen weiterhin vorhanden	positiv
TF-28	Dashboard-Kennzahlen nach Änderung	Anwendung gestartet	Neue Immobilie mit Mieteinheit anlegen, zum Dashboard wechseln	Kennzahlen zu Immobilien und Einheiten berücksichtigen den neu angelegten Datensatz	positiv

4 Performance-Test

Das Anforderungsdokument definiert für die Performance einen Schwellwert von 100 ms Reaktionszeit bei einem Bestand von bis zu 20 Mieteinheiten und legt für die Skalierbarkeit einen Gesamtbestand von bis zu 200 Mietverträgen sowie 400 Kostenpositionen fest. Das Skript `scripts/performance_benchmark.py` füllt eine SQLite-Datenbank exakt mit dieser Datenmenge und misst die Ausführungszeit der Service-Schicht hinter den zentralen Bedieninteraktionen, da diese und nicht das Zeichnen der ohnehin kleinen Ergebnistabellen den bestimmenden Anteil der Reaktionszeit ausmacht. Jede Operation wird mehrfach wiederholt gemessen, um einzelne Ausreißer durch Betriebssystem-Jitter nicht überzubewerten.

Operation	Ø-Zeit	Max.-Zeit
Dashboard-Refresh (alle Kennzahlen)	5,1 ms	9,9 ms
Mietverträge-Tab laden (alle 200)	3,6 ms	4,7 ms
Kostenpositionen-Tab laden (1 Immobilie, 80)	1,8 ms	3,3 ms
Abrechnung erstellen (1 Jahr, 4 Verträge)	23,6 ms	62,4 ms
Abrechnung erstellen (Worst Case, 10-Jahres-Zeitraum)	41,6 ms	66,7 ms
Abrechnungsverlauf laden	0,8 ms	3,6 ms

Tabelle 4.1: Ergebnis des Performance-Benchmarks bei maximaler dokumentierter Skalierung

Sämtliche gemessenen Operationen bleiben mit maximal 66,7 ms deutlich unter dem geforderten Schwellwert von 100 ms, selbst im konstruierten Worst-Case-Szenario eines zehnjährigen statt des realistischen einjährigen Abrechnungszeitraums. Damit ist die im Anforderungsdokument formulierte Performance-Anforderung bei der dort ebenfalls definierten Skalierungsgrenze erfüllt. Die Messung deckt die Datenbank- und Berechnungslogik der Service-Schicht ab, nicht das clientseitige Zeichnen der Qt-Tabellen. Da die betroffenen Ansichten höchstens 200 Zeilen darstellen, fällt der zusätzliche Rendering-Anteil gering aus und ändert nichts an der Einhaltung des Schwellwerts.

5 Fazit

Dieses Dokument hat mit 42 automatisierten Unit-Tests bei 98–100 % Testabdeckung der Service-Schicht, 28 manuellen Systemtestfällen, die alle Geschäftsregeln und zentralen Dialoge sowohl im positiven als auch im negativen Fall abdecken, sowie dem Performance-Benchmark aus Abschnitt 4 sowohl die im Anforderungs- und Spezifikationsdokument definierten funktionalen Anforderungen und Geschäftsregeln als auch die quantifizierten nichtfunktionalen Anforderungen an Performance und Skalierbarkeit verifiziert. Alle Tests und der Benchmark bestehen zum Zeitpunkt der Erstellung dieses Dokuments reproduzierbar unterhalb des geforderten Schwellwerts, sodass die im angestrebten Ergebnis der Projektdokumentation formulierte Zielsetzung einer funktionsfähigen, den Anforderungen entsprechenden Anwendung erreicht ist.

Projekt: Software Engineering

Abstract

MietManager

Portfolio von Michael Bangas

Matrikelnummer: IU14132885

Kursbezeichnung: Projekt: Software Engineering (DLMCSPSE01_D)

Studiengang: M.Sc. Informatik

IU Internationale Hochschule

25. September 2026

Making-of. MietManager ist eine PyQt6-Desktop-Anwendung, mit der private Kleinvermieter ihre Immobilien, Mieter und Mietverträge verwalten und daraus automatisiert eine versandfertige Nebenkostenabrechnung als PDF-Geschäftsbrief erzeugen. Der Zeitplan der Projektdokumentation sah für das Gesamtprojekt vier Kalenderwochen vom 27.07.2026 bis zum 23.08.2026 vor. Tatsächlich zog sich die Fertigstellung bis zum 26.09.2026, weil die Bearbeitung neben einem Vollzeit-Studium und einer Berufstätigkeit nur abends und am Wochenende möglich war. Der Kern der Implementierung fand dagegen konzentriert innerhalb einer Woche statt, wie sich anhand der Commit-Historie nachvollziehen lässt.

Vor der eigentlichen Implementierung stand die Konzeptionsphase mit Anforderungs- und Spezifikationsdokument. Für die Benutzerschnittstellen entstanden dabei zunächst handskizzierte und daraus abgeleitete digitale GUI-Mockups der zentralen Ansichten, darunter das Dashboard, die tabellarischen Datenansichten für Immobilien, Mieter, Mietverträge und Kostenpositionen, der Nebenkostenabrechnung-Tab und das Profil-Popup. Diese Mockups legten Struktur und Aufbau der wichtigsten Dialoge bereits vor dem ersten Code fest und dienten während der Implementierung als Vorlage für die tatsächliche Oberfläche.

Nach dieser Planungsphase bildete das Grundgerüst mit Projekt-Setup, das SQLAlchemy-Datenmodell für die acht Geschäftsobjekte sowie Datenbank-Engine, Session-Handling und ein Seed für Testdaten. Anschließend folgten die Geschäftsregeln und die Berechnungslogik der Nebenkostenabrechnung, bewusst vor der Oberfläche und direkt mit ersten automatisierten pytest-Tests abgesichert, sodass die Service-Schicht von Beginn an unabhängig von Qt testbar war. Am 14.08.2026 entstand die grafische Oberfläche, wobei zunächst Hauptfenster, Navigation und Theming und danach die CRUD-Tabs und -Dialoge für Immobilien, Mieteinheiten, Mieter und Mietverträge und anschließend der Nebenkostenabrechnung-Tab mit Vermieterprofil erstellt wurden. Anschließend wurde die Anwendung gegen Fehler gehärtet. Sie wurde als eigenständige Windows-.exe mit PyInstaller gebaut, die automatisierten Tests wurden auf rund 100 % Abdeckung der Service-Schicht ausgeweitet, ein Absturz beim Löschen eines bereits abgerechneten Mietvertrags wurde behoben und mehrere UX-Schwächen wurden ausgeräumt, wie in der anschließenden Reflexion erläutert.

Reflexion. Die frühe Entscheidung für eine strikte Schichtentrennung zahlte sich aus. Als sich zeigte, dass Geschäftsregeln zunächst nur innerhalb der UI-Schicht geprüft wurden, ließ sich die Logik ohne Umbau der Oberfläche in das bereits bestehende, von Qt und Datenbank unabhängige Service-Modul auslagern und dort automatisiert testen.

Mehrere Probleme zeigten sich erst während der Nutzung der eigenen, bereits laufenden Anwendung. So traten wiederholt visuelle Fehler in der Benutzeroberfläche auf. Deutlich wurde außerdem, dass die anfangs in jedem Tab vorhandenen manuellen Aktualisieren-Schaltflächen die Bedienung unnötig

erschweren. Nutzer mussten nach jeder Änderung selbst aktiv werden, damit sie an anderer Stelle sichtbar wurde. Als Reaktion darauf wurden diese Schaltflächen in allen Tabs entfernt und durch eine automatische Aktualisierung beim Tabwechsel ersetzt, ergänzt um eine optische Kennzeichnung nicht veränderbarer Felder durch Grautöne. Eine Anpassung, die im ursprünglichen Anforderungs- und Spezifikationsdokument nicht vorgesehen war, sondern sich erst aus der Erfahrung mit der eigenen fertigen Anwendung ergab.

Der ursprünglich geplante Zeitplan wurde nicht eingehalten. Statt der vorgesehenen vier Wochen benötigte das Projekt bis zum 26.09.2026 mehr als das Doppelte an Kalenderzeit. Ausschlaggebend war vor allem, dass die Bearbeitung neben Studium und Beruf nur in begrenzten Zeitfenstern abends und am Wochenende stattfinden konnte. Diese engen Zeitfenster verstärkten auch die im Risikomanagement der Projektdokumentation als R2 identifizierte Wartezeit auf Tutor-Feedback. Traf eine Rückmeldung etwa an einem Wochentag ein, ließ sie sich häufig erst im nächsten freien Abend- oder Wochenendfenster bearbeiten. So summierten sich die begrenzte eigene Zeit und die Wartezeiten zu einer deutlich längeren Kalenderlaufzeit, als es die anhand der Commit-Historie sichtbare, konzentrierte Implementierungswoche vermuten ließe.

Die wichtigste Erkenntnis betrifft daher die Zeitplanung selbst. Ein Zeitplan, der im Wesentlichen die eigene Bearbeitungszeit kalkuliert, unterschätzt systematisch den tatsächlichen Kalenderaufwand, wenn echte Arbeitszeit nur in schmalen Zeitfenstern neben Studium und Beruf zur Verfügung steht. Künftig sollte ein Zeitplan diese begrenzte Verfügbarkeit von Anfang an realistisch abbilden, statt von durchgehend nutzbaren Arbeitstagen auszugehen. Zweitens hat sich bestätigt, dass Testbarkeit von Beginn an in der Architektur mitgedacht werden sollte, statt sie nachträglich in eine bereits bestehende UI-Schicht hineinzuziehen. Die Implementierungsreihenfolge dieses Projekts, zuerst Geschäftslogik und Tests und erst danach die Oberfläche, hat das bereits weitgehend erfüllt. Drittens zeigte sich, dass Usability-Schwächen wie die anfänglichen manuellen Aktualisieren-Schaltflächen oft erst durch tatsächliche Nutzung der eigenen Anwendung sichtbar werden, weshalb eigenes Testen der fertigen Oberfläche fester Bestandteil des Entwicklungsprozesses sein sollte, statt sich allein auf die vorab entworfenen Mock-ups zu verlassen.

Für eine Weiterentwicklung über dieses Portfolio hinaus böten sich mehrere Erweiterungen an. Kurzfristig ließen sich die bereits als Kann eingestufteten Funktionen automatischer E-Mail-Versand und CSV-Export der Dashboard-Kennzahlen ohne Änderung der bestehenden Architektur ergänzen. Bei wachsendem Kundenstamm über die dokumentierte Skalierungsgrenze hinaus oder bei Bedarf an Mehrbenutzerbetrieb müsste die dateibasierte SQLite-Anbindung durch eine Client-Server-Datenbank ersetzt werden, wofür die bestehende Trennung von Service- und Datenzugriffsschicht bereits eine geeignete Grundlage bietet.